

Group Members:

Syed Sheharyar Tariq (25F-AI-077)

Huzefa Javed (25F-AI-093)

Aleena Khan (25F-AI-099)

Date: _____

Lab 1: Introduction to Digital Logic, Number Systems, and Logic Gates**1. Objectives**

After completing this lab, students will be able to:

- Understand the difference between analog and digital systems
- Represent numbers in binary, octal, and hexadecimal number systems
- Convert numbers between different number systems
- Identify basic logic gates and their symbols
- Verify the operation of logic gates using truth tables
- Implement basic logic gates using simple hardware or simulation tools

2. Background Theory**2.1 Analog vs Digital Signals**

Analog signals vary continuously with time, whereas digital signals have discrete values, typically represented using two levels: logic 0 (LOW) and logic 1 (HIGH). Digital systems are more robust to noise and are widely used in modern electronic systems.

2.2 Number Systems

Digital systems operate using the **binary number system (base-2)**. Other commonly used number systems include:

- **Decimal (base-10):** Digits 0–9
- **Binary (base-2):** Digits 0, 1
- **Octal (base-8):** Digits 0–7
- **Hexadecimal (base-16):** Digits 0–9 and A–F

Conversions between these systems are essential for understanding digital hardware and software.

2.3 Logic Gates

Logic gates are the fundamental building blocks of digital circuits. In electronics, a logic gate is an idealized or physical device implementing a Boolean function; that is, it performs a logical operation on one or more logical inputs, and produces a single logical output. Logic gates are primarily implemented using diodes or transistors acting as electronic switches, but can also be constructed using electromagnetic relays (relay logic), fluidic logic, pneumatic logic, optics, molecules, or even mechanical elements.

For small-scale logic, designers now use prefabricated logic gates from families of devices such as the TTL 7400 series by Texas Instruments, the CMOS 4000 series by RCA, and their more recent descendants. Increasingly, these fixed-function logic gates are being replaced by programmable logic devices, which allow designers to pack a large number of mixed logic gates into a single integrated circuit.

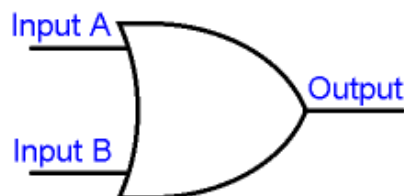
Logic circuits have one or more inputs and only one output. The circuits respond to various input combinations, and a truth table shows this relationship between circuits input combinations and its output. The truth table for a particular circuit explains how the circuit behaves under normal conditions.

The most basic logic gates include:

- AND
- OR
- NOT
- NAND
- NOR
- XOR
- XNOR

Each logic gate performs a specific Boolean operation and can be represented using symbols, Boolean expressions, and truth tables.

OR gate:



3. Equipment / Software Required

- Digital Logic Trainer Kit **or**
- Breadboard, ICs (7408 AND, 7432 OR, 7404 NOT – optional)
- Connecting wires

- Power supply (5V)
- **OR** Logic simulation software (e.g., Logisim / Multisim / Deeds / Proteus)

4. Pre-Lab Tasks

1. Convert the following numbers:
 - $(25)_{10}$ to binary and hexadecimal
 - $(101101)_2$ to decimal
 - $(3F)_{16}$ to decimal
2. Write the Boolean expressions for AND, OR, and NOT gates.
3. Draw logic symbols for AND, OR, and NOT gates.

5. Lab Procedure

Part A: Verification of Logic Gates

1. Construct the AND gate circuit using hardware or simulation software.
2. Apply all possible input combinations.
3. Observe and record the output.
4. Repeat steps 1–3 for OR and NOT gates.

Part B: Truth Table Verification

Verify the truth tables given below by practical implementation.

AND Gate

$$A \ B \ Y = A \cdot B$$

0 0 0

0 1 0

1 0 0

1 1 1

OR Gate

$$A \ B \ Y = A + B$$

0 0 0

0 1 1

$$A \oplus B = A + B$$

1 0 1

1 1 1

NOT Gate

$$A \oplus Y = A'$$

0 1

1 0

6. Observations

Students should record the observed outputs and compare them with the theoretical truth tables.

7. Results and Discussion

- Were the practical results consistent with the theoretical truth tables?
- Discuss possible reasons for any discrepancies (if observed).

8. Lab Assignment

1. Why is binary number system used in digital circuits?

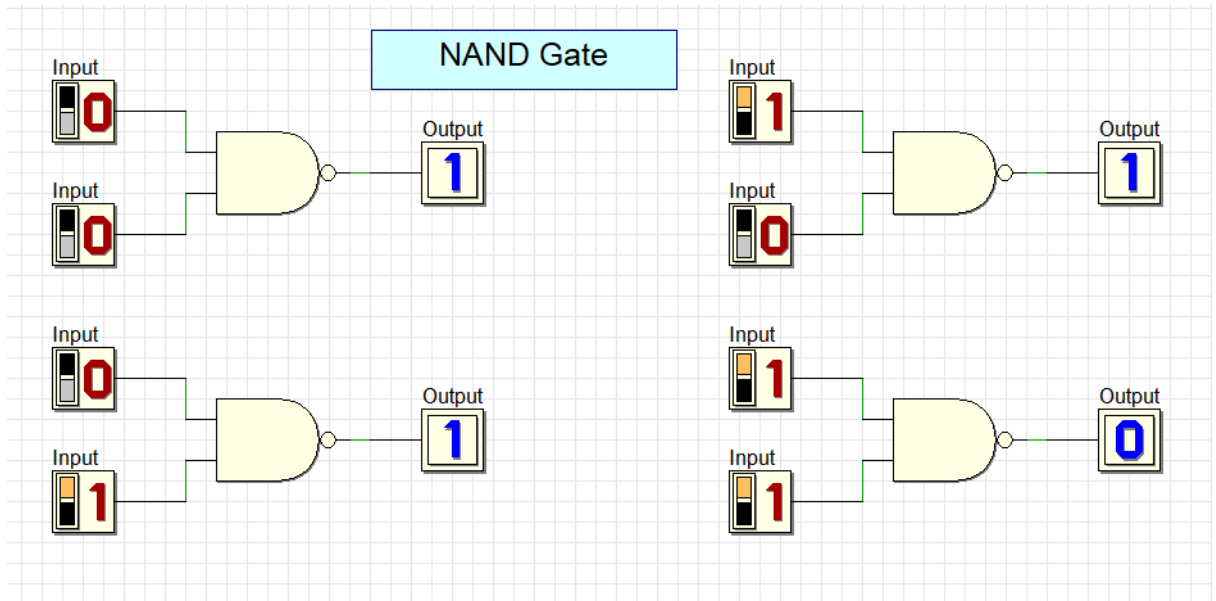
Answer:

Binary number system is used in digital circuits because electronic components like transistors operate in two stable states (ON and OFF), which can be easily represented as 1 and 0, making circuits simpler, more reliable, less affected by noise, faster to process, and efficient for data storage and logical operations.

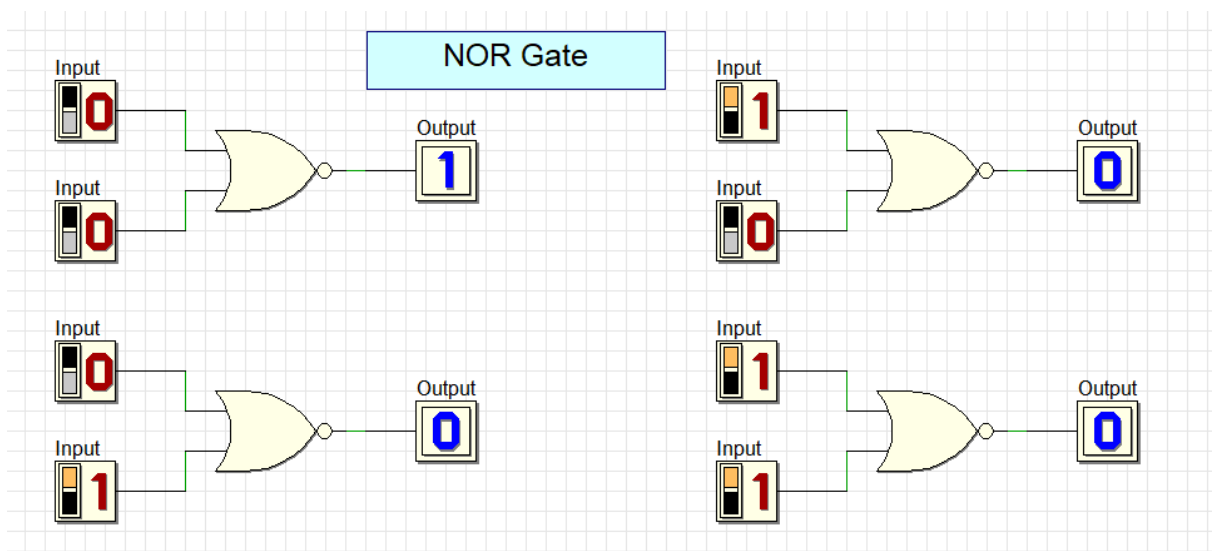
2. What happens if an input of a NOT gate is left floating?

Answer:

1. A floating input is **not connected** to voltage or ground, so it can **pick up noise** from nearby circuits.
 2. The NOT gate might **read it as 0 or 1 randomly**, causing the **output to fluctuate**.
 3. This can lead to **erratic behaviour** in the circuit, which is **unreliable** and may cause errors.
3. Implement NAND and NOR gates and verify their truth tables.



A	B	A AND B	NAND (NOT A AND B)
0	0	0	1
0	1	0	1
1	0	0	1
1	1	1	0



A	B	A OR B	NOR (NOT A OR B)
0	0	0	1
0	1	1	0
1	0	1	0
1	1	1	0

4. Convert $(45)_{10}$ into binary and hexadecimal.

1 Decimal to Binary

We divide the number by 2 repeatedly and write down the remainders.

Step	Divide by 2	Quotient	Remainder
1	$45 \div 2$	22	1
2	$22 \div 2$	11	0
3	$11 \div 2$	5	1
4	$5 \div 2$	2	1
5	$2 \div 2$	1	0
6	$1 \div 2$	0	1

Now, write **remainders from bottom to top**:

$(101101)_2$

✓ Binary: **101101**

2 Decimal to Hexadecimal

We divide the number by 16 repeatedly.

Step	Divide by 16	Quotient	Remainder
1	$45 \div 16$	2	13

- 13 in hexadecimal = **D**
- Quotient = 2 → write as is

$$(2D)_{16}$$

✓ Hexadecimal: **2D**

✓ **Final Answer:**

- Binary: **101101₂**
- Hexadecimal: **2D₁₆**

WRITE CONCLUSION (What have you learnt from this experiment?)

From this experiment, I have learned the fundamental differences between analogue signals (continuous) and digital signals (discrete). I have gained proficiency in converting values between binary, octal, decimal, and hexadecimal systems, which is essential for understanding digital hardware. Additionally, I have verified that logic gates serve as the building blocks of digital systems, each performing a specific Boolean operation that can be predicted by a truth table and implemented using integrated circuits or simulation tools.

Group Members:**Syed Sheharyar Tariq (25F-AI-077)****Huzefa Javed (25F-AI-093)****Aleena Khan (25F-AI-099)****Date:** _____**Lab # 02 TO UNDERSTAND THE CONCEPT OF UNIVERSAL NAND GATE IN DIGITAL CIRCUIT DESIGN****OBJECTIVES:**

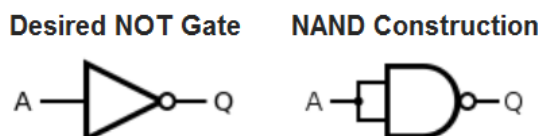
- 01: To understand the concept of universal NAND Gate
- 02: To implement Logic Circuits using only NAND gates.
- 03: Use the **DEEDS** simulation environment to design and verify digital circuits
- 04: Analyze truth tables and simulation results for correctness

BACKGROUND THEORY

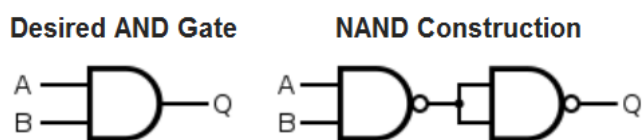
Because the NAND function has functional completeness all logic systems can be converted into NAND gates. This is also true for NOR gates. In principle, any combinatorial logic function can be realized with enough NAND gates and/or NOR Gates.

NAND GATE EQUIVALENT OF BASIC LOGIC GATE**NOT Gate:**

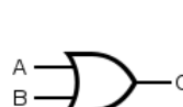
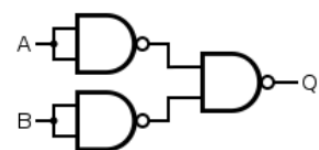
A NOT gate is made by joining the inputs of a NAND gate together. Since a NAND gate is equivalent to an AND gate followed by a NOT gate, joining the inputs of a NAND gate leaves only the NOT gate.

**AND Gate:**

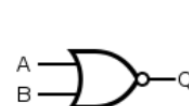
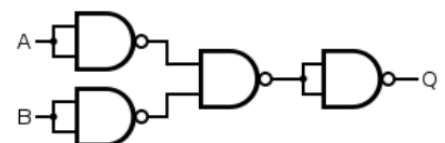
An AND gate is made by following a NAND gate with a NOT gate as shown below.

**OR Gate:**

If the truth table for a NAND gate is examined or by applying De Morgan's Laws, it can be seen that if any of the inputs are 0, then the output will be 1. To be an OR gate, however, the output must be 1 if any input is 1. Therefore, if the inputs are inverted, any high input will trigger a high output.

Desired OR Gate**NAND Construction****NOR Gate:**

A NOR gate is simply an inverted OR gate. Output is high when neither input A nor input B is high:

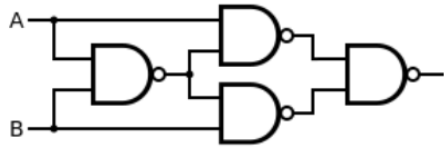
Desired NOR Gate**NAND Construction****XOR Gate:**

An XOR gate is constructed similarly to an OR gate, except with an additional NAND gate inserted such that if both inputs are high, the inputs to the final NAND gate will also be high, and the output will be low.

Desired XOR Gate



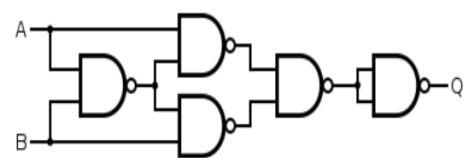
NAND Construction



Desired XNOR Gate



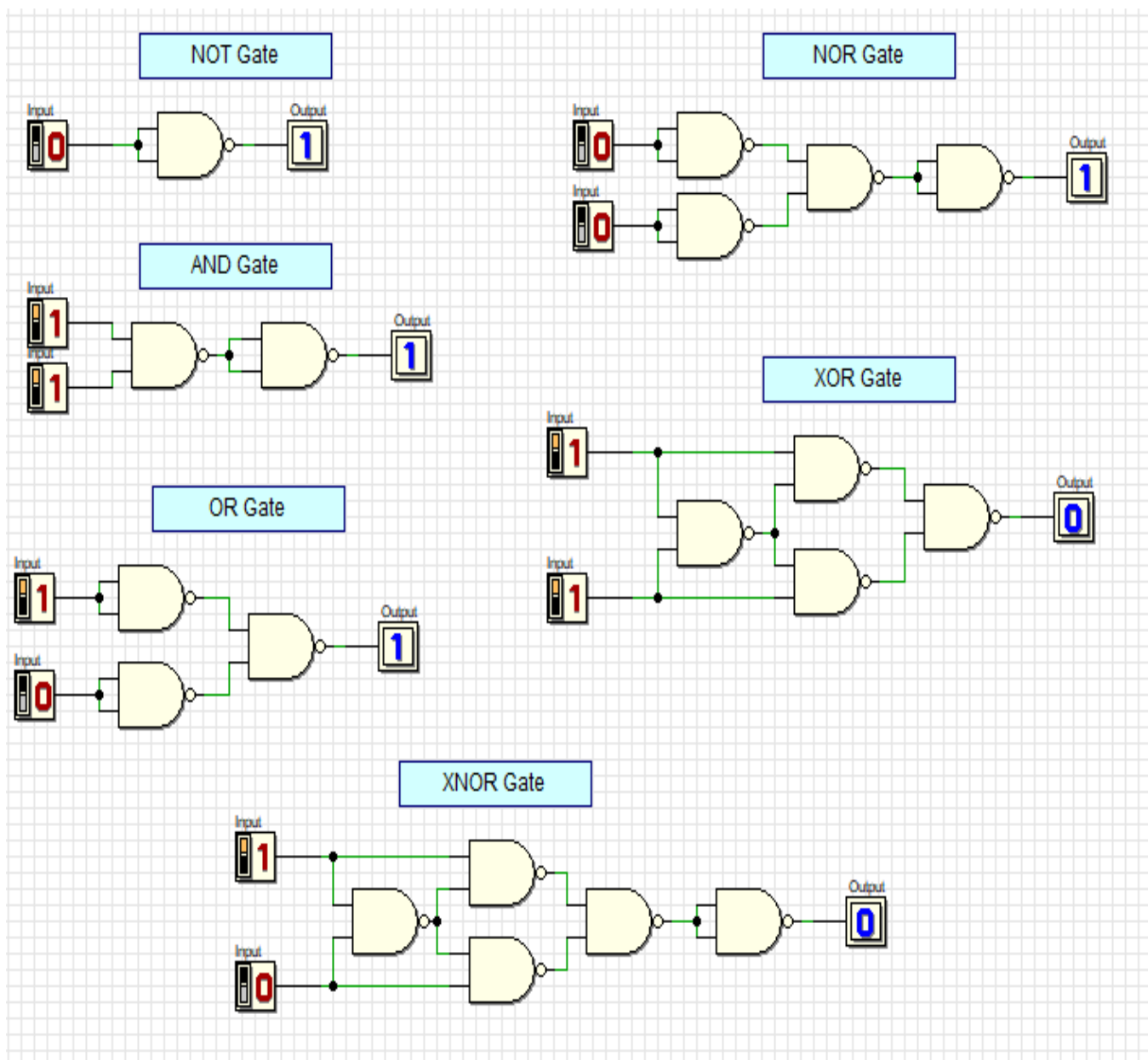
NAND Construction

**XNOR Gate:**

An XNOR gate is simply an XOR gate with an inverted output:

LAB TASK

1. Paste the screenshot of circuits made on Deeds.



2. Consider the Boolean expression given below:

$$X = A(B + C)$$

**Construct this circuit using NAND gates on simulator.
Paste the screenshot below.**

Also record the readings on the truth table below.

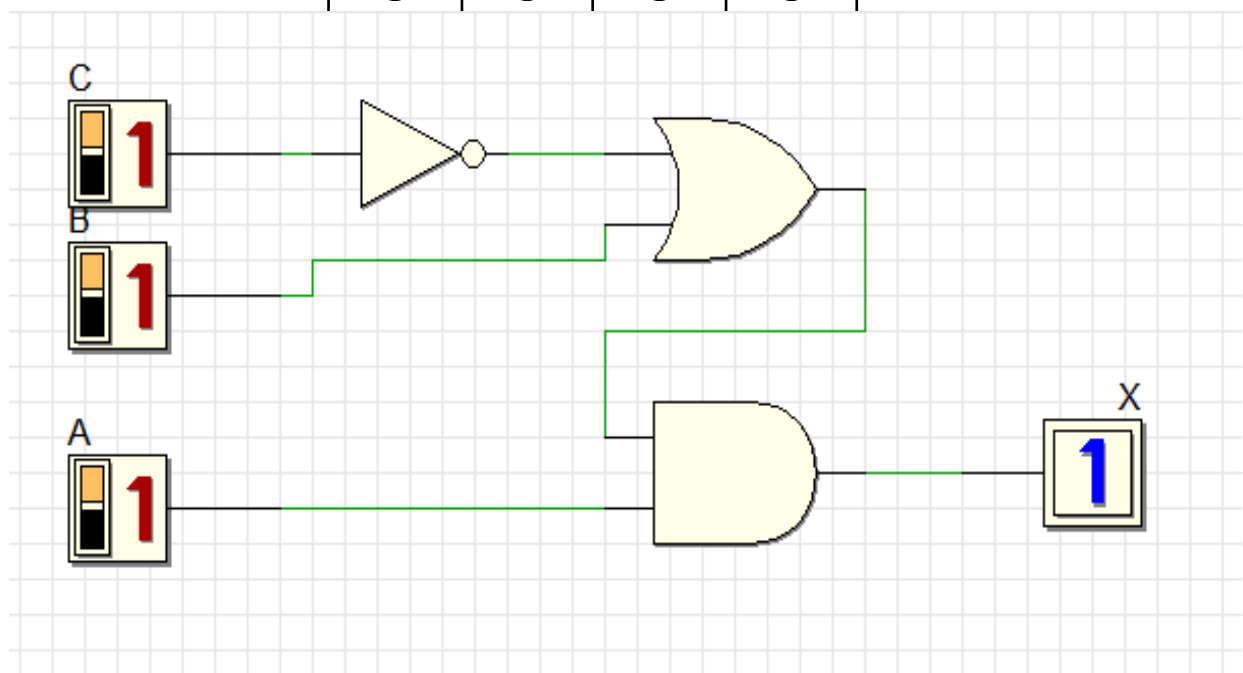
Table 2.1: Truth table

NAND Gate

0	0	0	0
0	0	1	0
0	1	0	0
0	1	1	0
1	0	0	1
1	0	1	0
1	1	0	1
1	1	1	1

for NAND equivalent
based circuit

equivalent of the given



expression:

CONCLUSION (What have you learnt from this experiment?):

From this experiment, I have learned that the **NAND gate** is a universal gate because it can replicate the functions of NOT, AND, and OR gates. By applying De Morgan's Laws and functional completeness, any complex Boolean expression, such as $X = AB + C$, can be implemented using exclusively NAND gates. Additionally, I practiced verifying logic designs

by comparing simulation results in **DEDS** against theoretical truth tables to ensure circuit correctness

Date: _____

Group Members:

Syed Sheharyar Tariq (25F-AI-077)

Huzefa Javed (25F-AI-093)

Aleena Khan (25F-AI-099)

Lab # 03 TO UNDERSTAND THE CONCEPT OF UNIVERSAL NOR GATE IN DIGITAL CIRCUIT DESIGN**OBJECTIVES:**

O1: To understand the concept of universal NOR Gates

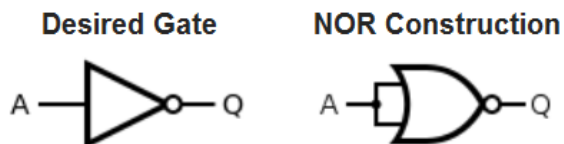
O2: To implement Logic Circuits using only NOR gates.

BACKGROUND THEORY

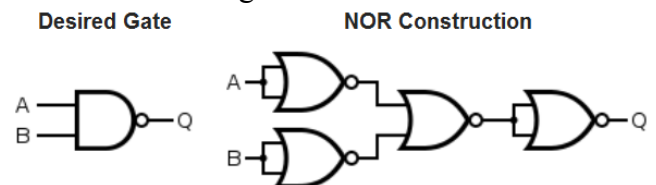
Because the NOR function has functional completeness all logic systems can be converted into NOR gates. In principle, any combinatorial logic function can be realized with enough NOR Gates, like NAND Gate.

NOR GATE EQUIVALENT OF BASIC LOGIC GATES**NOT Gate:**

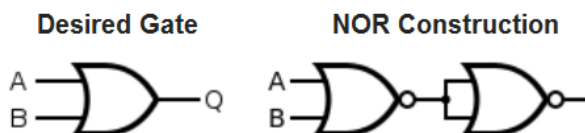
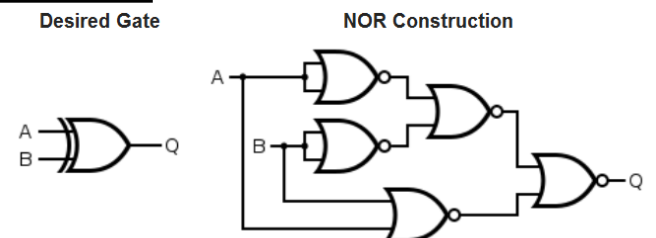
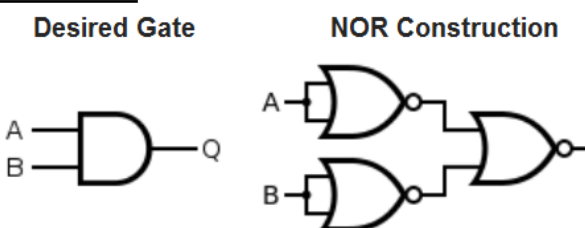
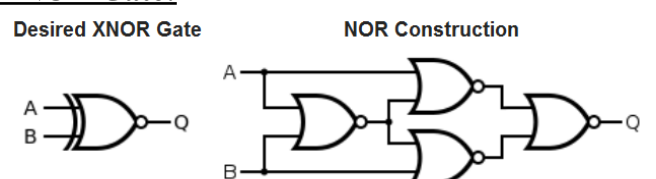
Same as that in NAND Logic, shorting the inputs of a NOR gate gets us a NOT gate:



A NAND gate is made using an AND gate in series with a NOT gate:

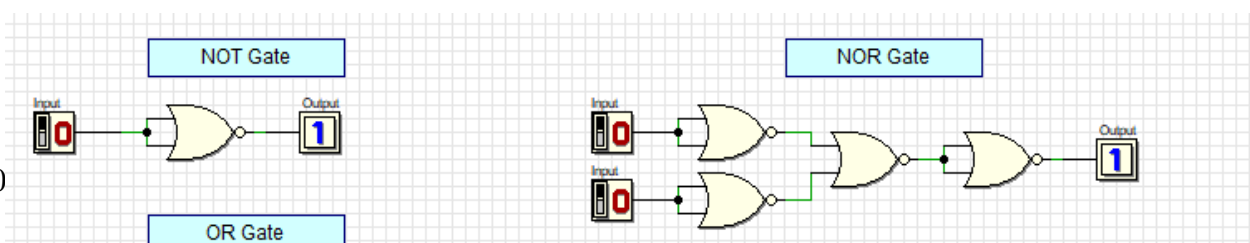
**OR Gate:**

The OR gate is simply a NOR gate followed by another NOR gate.

**XOR Gate:****AND Gate:****XNOR Gate:****LAB TASK****PROCEDURE:**

1. Construct the NOR equivalent circuits of above logic gates on Simulator and paste their screenshots below.

Lab # 0



2. Apply inputs and check corresponding Outputs
3. Design the equivalent circuit for the following expression using Universal NOR Gate;

$$X = \underline{AB} + C$$

Paste screenshot of NOR Gate equivalent of the given expression:

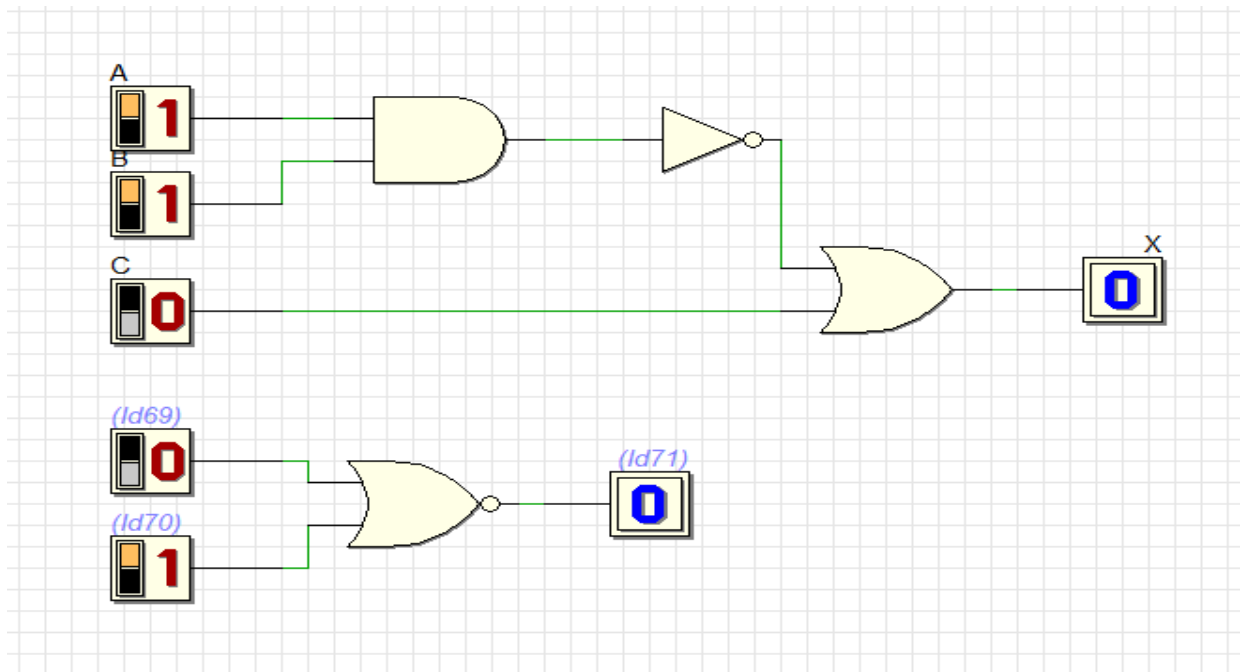


Table 3.1: Truth table

0	0	0	1
0	0	1	0
0	1	0	0
0	1	1	0
1	0	0	0
1	0	1	0
1	1	0	0
1	1	1	0

for NOR equivalent circuit.

CONCLUSION (What this experiment?):

The NOR gate is functionally complete, meaning any Boolean expression can be implemented without needing AND, OR, or NOT gates.

Implementing simple logic like an AND gate requires more physical NOR gates than a standard IC, but using a single gate type can simplify the manufacturing process of integrated circuits.

have you learnt from

AOI (AND-OR-Invert) logic can be systematically converted into universal gate equivalents by following specific substitution rules for each component.

LEARNING OUTCOMES

Upon successful completion of the lab, students will be able to:

LO1: Understand the importance of NAND and NOR Gates as Universal gates.

LO2: Apply the rules of converting AOI Gates to their Universal NAND or Universal NOR equivalents.

Group Members:**Syed Sheharyar (25F-AI-077)****Huzefa Javed (25F-AI-093)****Aleena Khan (25F-AI-099)****Date:** _____**Lab # 04 TO UNDERSTAND DE MORGAN'S THEOREM & BOOLEAN ALGEBRA USING DEEDS**

OBJECTIVES:

- Verify and understand concept of De Morgan's Theorems using simulation
- Understand and validate De Morgan's Theorem.

APPARATUS:

- PC/Laptop
- Deeds (Digital Electronics Education and Design Suite)

THEORY:

In propositional logic and Boolean algebra, De Morgan's laws are a pair of transformation rules that are both valid rules of inference. The rules can be expressed in English as:

"THE NEGATION OF A CONJUNCTION IS THE DISJUNCTION OF THE NEGATIONS."

AND

"THE NEGATION OF A DISJUNCTION IS THE CONJUNCTION OF THE NEGATIONS."

DeMorgan's theorem may be thought of in terms of breaking a long bar symbol. When a long bar is broken, the operation directly underneath the break changes from addition to multiplication, or vice versa, and the broken bar pieces remain over the individual variables.

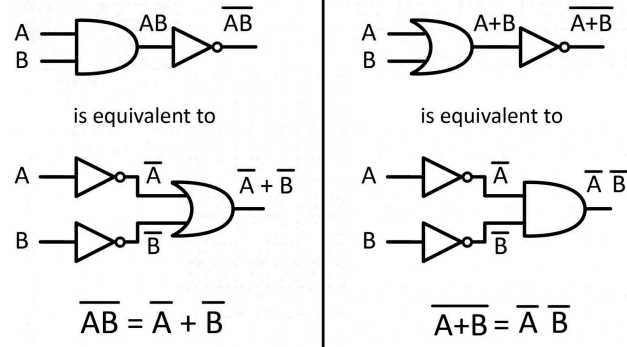
Mathematically, Demorgan's theorem laws are as follows:

1. $(A \cdot B)' = A' + B'$
2. $(A + B)' = A' \cdot B'$

DeMorgan's Theorems

Fig 3.1: Illustration of De Morgan's Theorem

Lab # 04



De Morgan's Laws

Fig 3.2: De Morgan's theorem equivalence

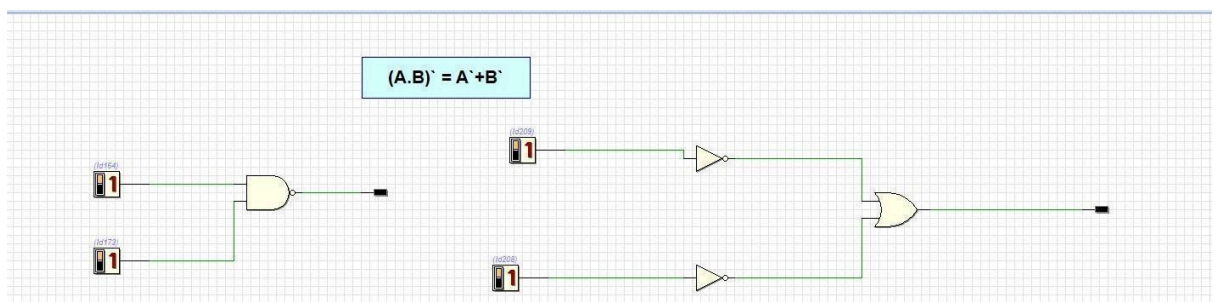
PROCEDURE:

1. Open Deeds software
2. Add AND, OR gates and inputs A, B, C
3. Observe output using LEDs
4. Build both sides and compare outputs

LAB TASK 1

A	B	$(A \cdot B)'$	$A' + B'$	Equal?
0	0	1	1	Yes
0	1	1	1	Yes
1	0	1	1	Yes
1	1	0	0	Yes

Paste screenshot below of theorem 1 implementation.



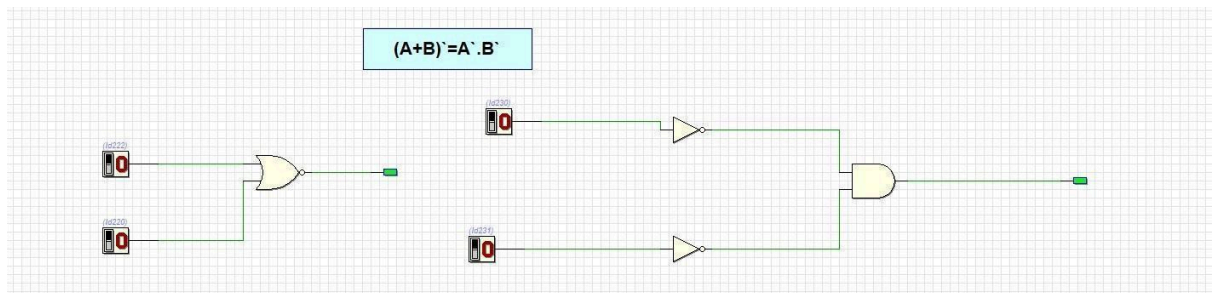
LAB TASK 2

A	B	$(A+B)'$	$A' \cdot B'$	Equal?
0	0	1	1	Yes
0	1	0	0	Yes
1	0	0	0	Yes

Lab # 04

1	1	0	0	Yes
---	---	---	---	-----

Paste screenshot below of theorem 2 implementation.



LAB TASK 3

Simplify the following Boolean expression and reduce it using DeMorgan's Theorems.

a. $Y = \underline{(A + BC)}$

Apply De Morgan's Theorem:

$$(A + X)' = A' \cdot X'$$

So:

$$Y = A' \cdot ((BC)')$$

Now:

$$((BC)')' = BC$$

Final Answer:

$$Y = A' \cdot BC$$

b. $Y = \underline{(A + B) \cdot (A + C)}$

Apply De Morgan:

$$(X \cdot Y)' = X' + Y' \quad Y = (A + B)' + (A + C)'$$

Now apply De Morgan again:

$$(A + B)' = A' \cdot B' \quad (A + C)' = A' \cdot C' \quad Y = A' \cdot B' \cdot A' \cdot C' = A' \cdot B' \cdot C'$$

So:

$$Y = A' B' C'$$

Factor A' :

$$Y = A' (B' + C')$$

Final Answer:

$$Y = A' (B' + C')$$

CONCLUSION (What have you learnt from this experiment?):

IN THIS EXPERIMENT, WE STUDIED AND VERIFIED **DE MORGAN'S THEOREMS** USING **DEEDS** SIMULATION. **WE** OBSERVED THAT THE COMPLEMENT OF **AND** OPERATION BECOMES **OR** OF COMPLEMENTS AND VICE VERSA. **THE** TRUTH TABLES CONFIRMED THAT BOTH SIDES OF THE THEOREMS ARE EQUIVALENT. **WE** ALSO LEARNED HOW TO SIMPLIFY **BOOLEAN** EXPRESSIONS USING **DE MORGAN'S LAWS**, WHICH HELPS IN DESIGNING SIMPLER AND EFFICIENT DIGITAL CIRCUITS

Date: _____

Group Members:

Syed Sheharyar Tariq (25F-AI-077)

Huzefa Javed (25F-AI-093)

Aleena Khan (25F-AI-099)

Lab # 05 TO UNDERSTAND THE WORKING OF A HALF ADDER AND A FULL ADDER

OBJECTIVES:

O1: To understand working of half adder.

O2: To understand working of full adder.

BACKGROUND THEORY

In electronics, an adder or summer is a digital circuit that performs addition of numbers. In many computers and other kinds of processors, adders are used not only in the arithmetic logic unit(s), but also in other parts of the processor, where they are used to calculate addresses, table indices, and similar operations.

• **Half Adder:**

The half adder adds two single binary digits A and B. It has two outputs, sum (S) and carry (C). The carry signal represents an overflow into the next digit of a multi-digit addition. The simplest half-adder design, pictured on the right, incorporates an XOR gate for S and an AND gate for C. With the addition of an OR gate to combine their carry outputs, two half adders can be combined to make a full adder.

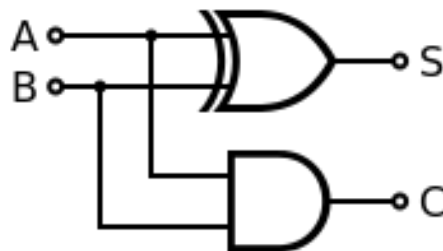


Fig 4.1: Half Adder Circuit
4.1: Truth Table

Table

0	0	0	0
0	1	1	0
1	0	1	0
1	1	0	1

Boolean Expressions: $Sum = A \oplus B$, $Carry = A \cdot B$

• **Full Adder:**

A full adder adds binary numbers and accounts for values carried in as well as out. A one-bit full adder adds three one-bit numbers, often written as A, B, and Carry in (C_{in}); A and B are the operands, and C_{in} is a bit carried in from the previous less significant stage. The full-adder is usually a component in a cascade of adders, which add 8, 16, 32, etc. bit binary numbers. The circuit produces a two-bit output, output carry and sum typically represented by the signals carry out (C_{out}) and Sum (S).

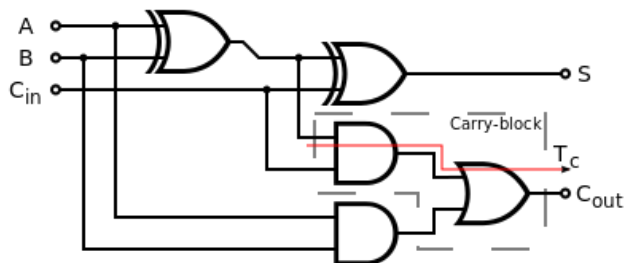


Fig 4.2: Full Adder Circuit

0	0	0	0	0
0	0	1	1	0
0	1	0	1	0
0	1	1	0	1
1	0	0	1	0
1	0	1	0	1
1	1	0	0	1
1	1	1	1	1

Table 4.2: Truth Table

Boolean Expressions:

$$Sum = A \oplus B \oplus C_{in}$$

$$C_{out} = (A \cdot B) + (C_{in} \cdot (A \oplus B))$$

PROCEDURE:

1. Construct the logic circuit for Half Adder in Digital simulator.
2. Apply all Input combinations to the circuit.
3. Check and note the outputs for the corresponding inputs and record the values in the observation table.
4. Now construct the circuit for Full Adder.
5. Repeat steps 2-3.

OBSERVATIONS:

Fig 4.3: Truth Table for Half Adder

Table 4.4: Truth Table for Full Adder

0	0	0		0	0	0	0	0	0
0	1	1		0	0	0	1	1	0
1	0	1		0	0	1	0	1	0
1	1	0		1	0	1	1	0	1
				1	0	0	0	1	0
				1	0	1	0	0	1
				1	1	0	0	0	1
				1	1	1	1	1	1

Q :) Mention an application both of Half Adder and Full Adder Circuit:

HALF ADDER APPLICATIONS:

- USED FOR ADDING TWO SINGLE BINARY DIGITS (BASIC ADDITION)
- USED IN SIMPLE CALCULATORS AND DIGITAL CIRCUITS
- BUILDING BLOCK FOR FULL ADDER

FULL ADDER APPLICATIONS:

- USED IN ARITHMETIC LOGIC UNIT (ALU) OF COMPUTERS
- USED FOR MULTI-BIT BINARY ADDITION (8-BIT, 16-BIT, ETC.)
- USED IN PROCESSORS, DIGITAL SYSTEMS, AND CALCULATORS

CONCLUSION (What have you learnt from this experiment?):

IN THIS EXPERIMENT, WE LEARNED HOW HALF ADDER AND FULL ADDER CIRCUITS WORK.

WE UNDERSTOOD HOW BINARY ADDITION IS PERFORMED USING LOGIC GATES LIKE XOR, AND, AND OR.

WE ALSO VERIFIED THE TRUTH TABLES BY APPLYING DIFFERENT INPUT COMBINATIONS.

THE HALF ADDER ADDS TWO BITS, WHILE THE FULL ADDER ADDS THREE BITS INCLUDING CARRY INPUT, MAKING IT MORE

USEFUL IN COMPLEX DIGITAL SYSTEMS.

LEARNING OUTCOMES

Upon successful completion of the lab, students will be able to:

LO1: Understand the concept of Half Adder and Full Adder.

LO2: To experimentally verify the behaviour of Half Adder and Full Adder

Name: Syed Sheharyar Tariq (25F-Ai-077)

Section: A2/B

Date: 24/04/2026

Lab # 06 TO UNDERSTAND THE WORKING OF A BINARY PARALLEL ADDER AND SUBTRACTOR

OBJECTIVES:

- O1: Design and implement a 4-bit binary parallel adder and subtractor
- O2: Understand how addition and subtraction are performed using the same circuit
- O3: Simulate the circuit using DEEDS or any digital simulator

BACKGROUND THEORY

Binary Addition

A parallel adder adds binary numbers using multiple full adders connected together. Each full adder adds two input bits and a carry input.

Binary Subtraction (2's Complement)

Subtraction is performed using 2's complement:

$$A - B = A + (2's \text{ complement of } B)$$

Steps:

1. Invert bits of B (1's complement)
2. Add 1
3. Add result to A

Combined Adder/Subtractor

A single circuit can perform both operations using XOR gates and a control input (Mode M):

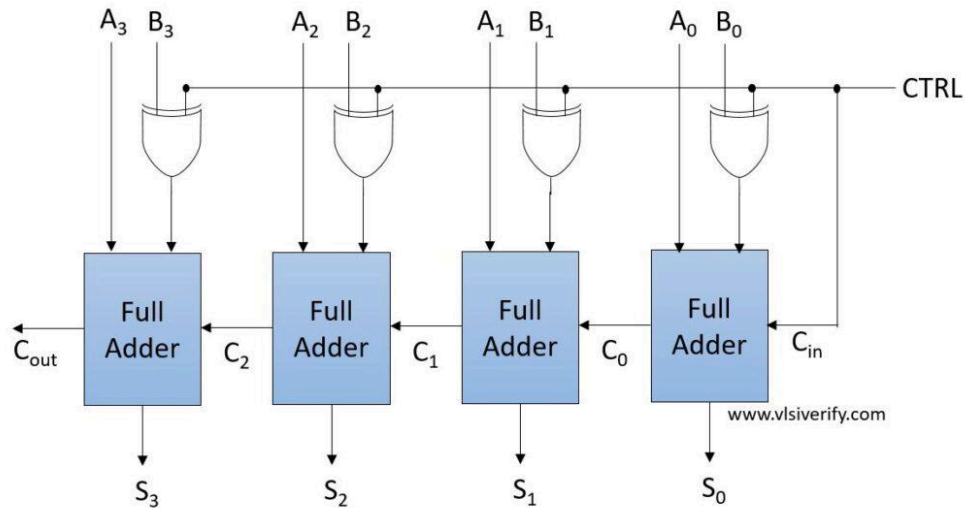
$M = 0 \rightarrow \text{Addition}$

$M = 1 \rightarrow \text{Subtraction}$

Logic:

$$B' = B \oplus M$$

$$\text{Carry-in} = M$$



4-Bit Adder Subtractor

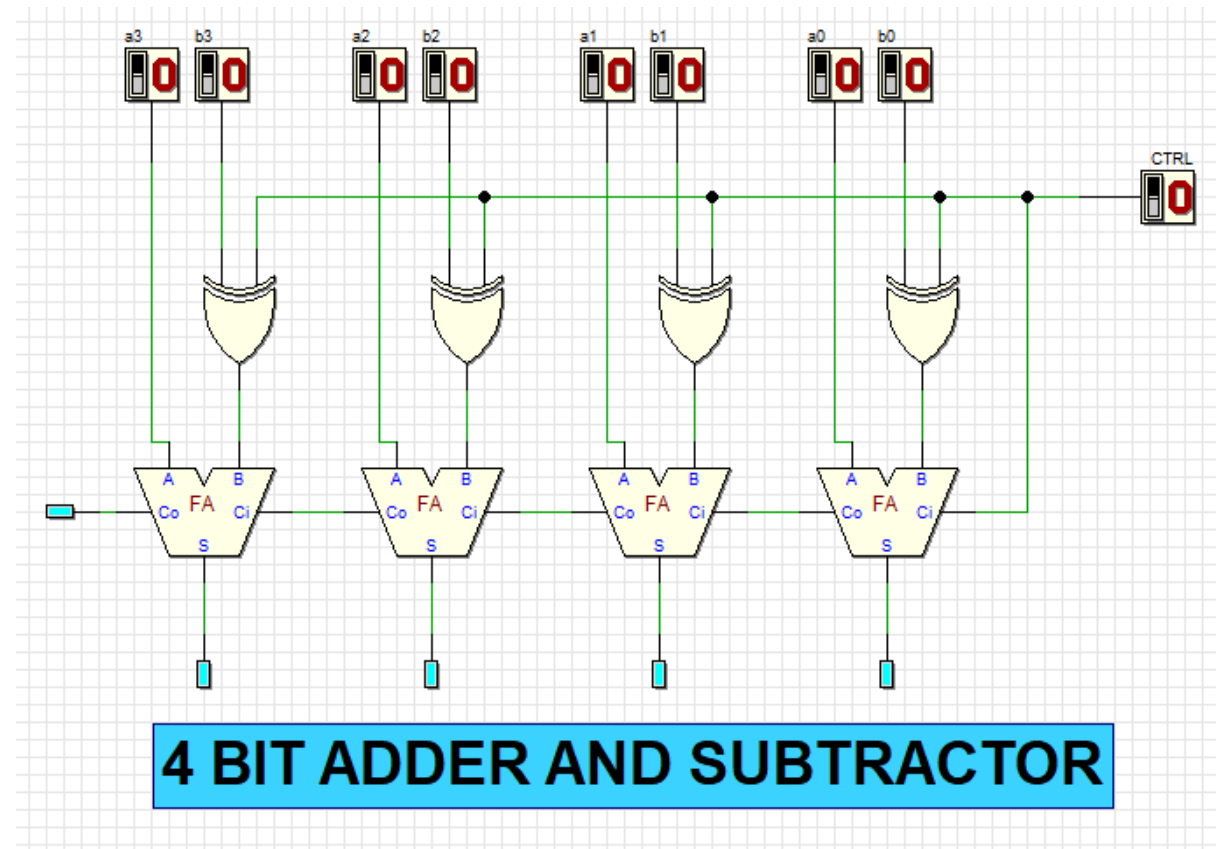
Required Components:

- 4 Full Adders
- 4 XOR Gates
- Switches (inputs)
- LEDs (outputs)
- Connecting wires

PROCEDURE:

1. Open DEEDS simulator
2. Place 4 full adders
3. Connect them in series
4. Add XOR gates before each B input
5. Connect XOR inputs to B and Mode (M)
6. Set Carry-in of first full adder = M
7. Connect outputs to LEDs
8. Add switches for A, B, and Mode

4-Bit Adder and Subtractor:



OBSERVATIONS:

A	B	M	OPERATION	RESULT
0101	0011	0	ADDITION	1000
0101	0101	1	SUBTRACTION	0000
1000	1010	0	ADDITION	0010 (CARRY= 1)
1000	1010	1	SUBTRACTION	1110

Q :) What is the role of XOR in this circuit?

Ans: XOR gates are used to conditionally invert the input B based on the mode (M). When $M = 0$, B remains unchanged for addition. When $M = 1$, XOR inverts B, which helps perform subtraction using 2's complement.

Q: WHY IS CARRY-IN EQUAL TO MODE?

Ans: Carry-in is equal to M to complete the 2's complement operation during subtraction. When $M = 1$, carry-in becomes 1 to add the required "+1". When $M = 0$, carry-in is 0 for normal addition.

Q: WHAT IS 2'S COMPLEMENT?

Ans: 2's complement is a method to represent negative binary numbers. It is obtained by inverting all bits (1's complement) and adding 1. It is used to perform subtraction using addition circuits.

CONCLUSION (What have you learnt from this experiment?):

Ans: This experiment demonstrated the working of a 4-bit adder-subtractor circuit using full adders and XOR gates. We learned how a single circuit can perform both addition and subtraction by using a control signal (Mode). The XOR gates help in conditional inversion of inputs, and the carry-in enables 2's complement operation for subtraction. This design is efficient and widely used in digital systems for arithmetic operations.

Group Members:

Syed Sheharyar Tariq (25F-AI-077)

Huzefa Javed (25F-AI-093)

Aleena Khan (25F-AI-099)

Date: 25 – 04 -2026

Lab # 07 A COMPREHENSIVE REVIEW OF COMBINATIONAL CIRCUIT DESIGN USING REAL LIFE

OBJECTIVES:

- O1: To apply the knowledge of Combinational Logic design for solving Real World Scenarios.
- O2: To analyse the usefulness of Digital circuits in handling complex problems

BACKGROUND THEORY

After getting considerable knowledge in the field of Combinational circuits design and their simplification using Boolean algebra and DeMorgan's theorem, it would be of very little use if we don't apply them to solve various Real world situations and to perform specific tasks.

LAB TASKS

The main target of the current lab is to study a Real World problem and to perform specific tasks by designing appropriate Combinational circuits. For the following given situation, design appropriate Combinational circuits to perform the required functions and implement the designed circuits on Deeds or any digital simulator.

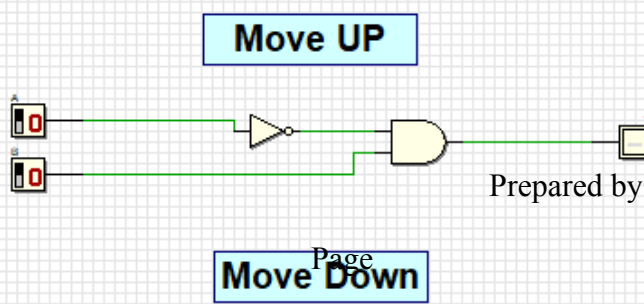
SCENARIO:

SCENARIO 1

(Attach working on a separate sheet)

Truth table for the system:**1**

A	B	Move Up = $A'B$	Move Down = AB'	Open Door = $AB + A'B'$
0	0	0	0	1
0	1	1	0	0
1	0	0	1	0
1	1	0	0	1

LOGIC EXPRESSION:**Move Up:** $A'B$ **Move Down:** AB' **Open Door:** $AB + A'B'$ **Logic gate implementation of the system****ELEVATOR CONTROLLER FOR 2 FLOORS (Ground Floor & First Floor)**

CONCLUSION (What have you learnt from this experiment?):

Ans: In this experiment, we learned how to design combinational circuits using Boolean expressions and truth tables. We applied our knowledge to a real-life type problem and created logical expressions for different outputs like move up, move down, and open door. We also understood how logic gates can be used to implement these expressions. This lab helped us see how digital circuits can solve practical problems and made our understanding of combinational logic clearer.

2

NAME: Syed Sheharyar Tariq

ROLL NO: 25F-AI-077

SECTION: A2/B

Instructor: Miss Bushra

Date: 8 May 2026

Lab # 08 To understand the working of binary multiplier circuit

Objectives:

- O1: To design and implement a 4-bit binary multiplier
- O2: To understand binary multiplication using logic circuits
- O3: To simulate using DEEDS or any digital simulator

Background Theory

Binary multiplication is similar to decimal multiplication but uses only 0s and 1s. Each bit of the multiplier is ANDed with the multiplicand, and results are shifted and added. For a 4-bit multiplier:

If $A = A_3 A_2 A_1 A_0$ and $B = B_3 B_2 B_1 B_0$,

Partial products are generated using AND gates and then added using adders to get the final product. The result (product) is an **8-bit output**:

The multiplier consists of:

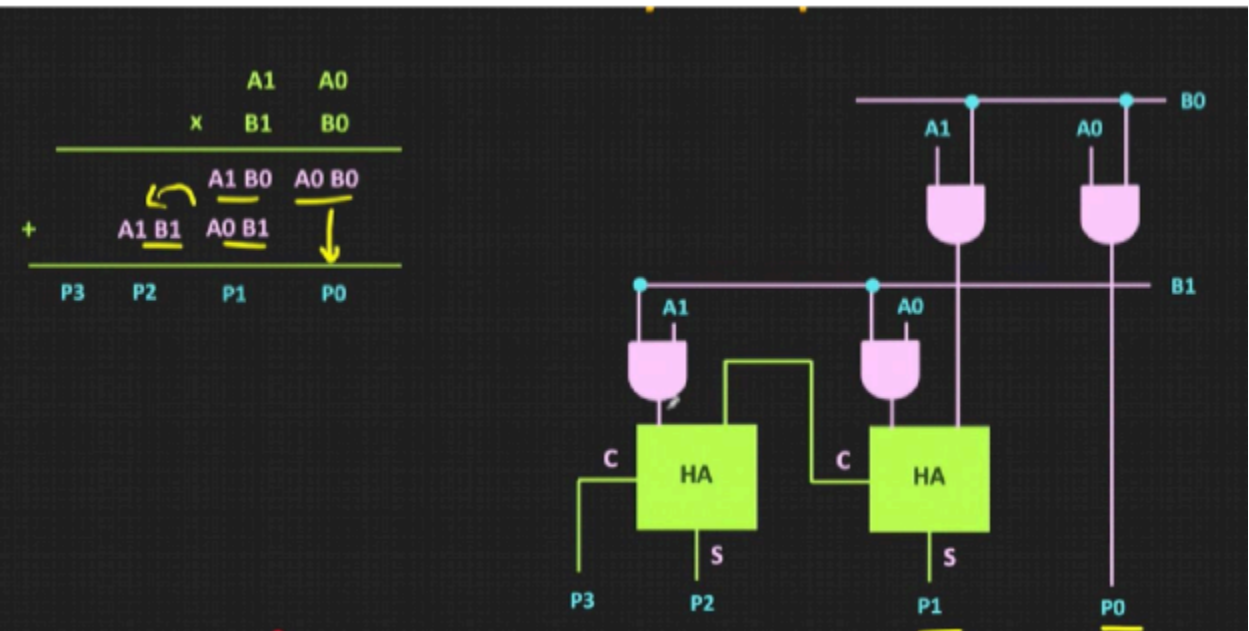
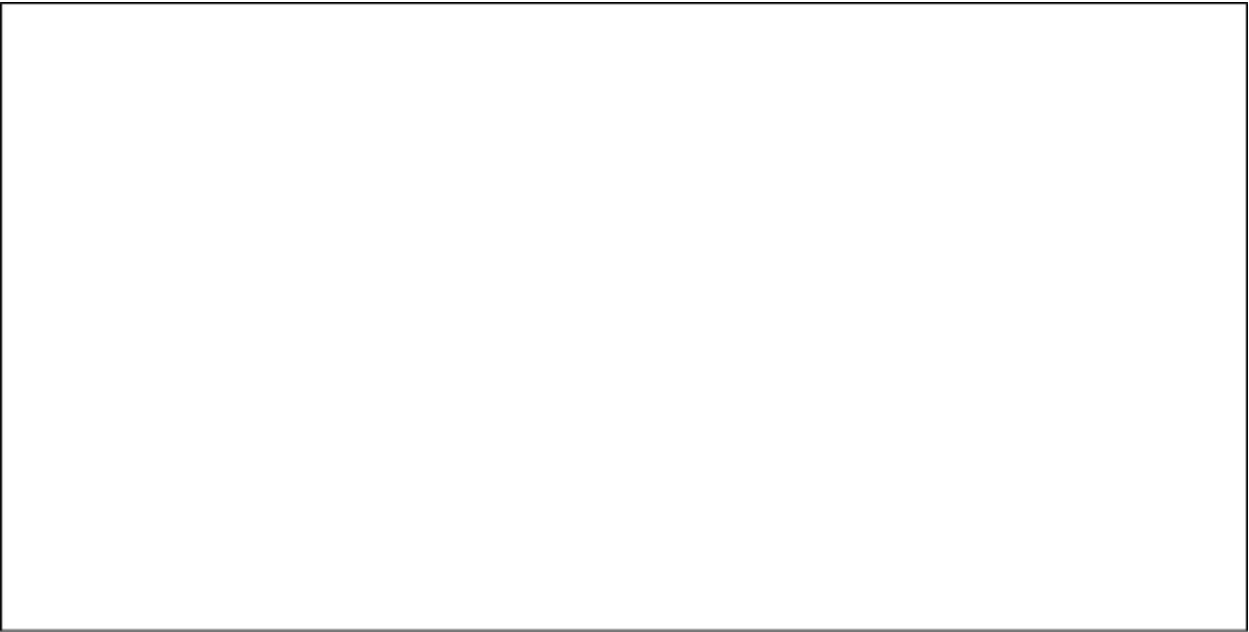
- **AND gates** generate partial products
- **Adders (Half→ and Full Adders)** → sum partial products

Partial Product Generation

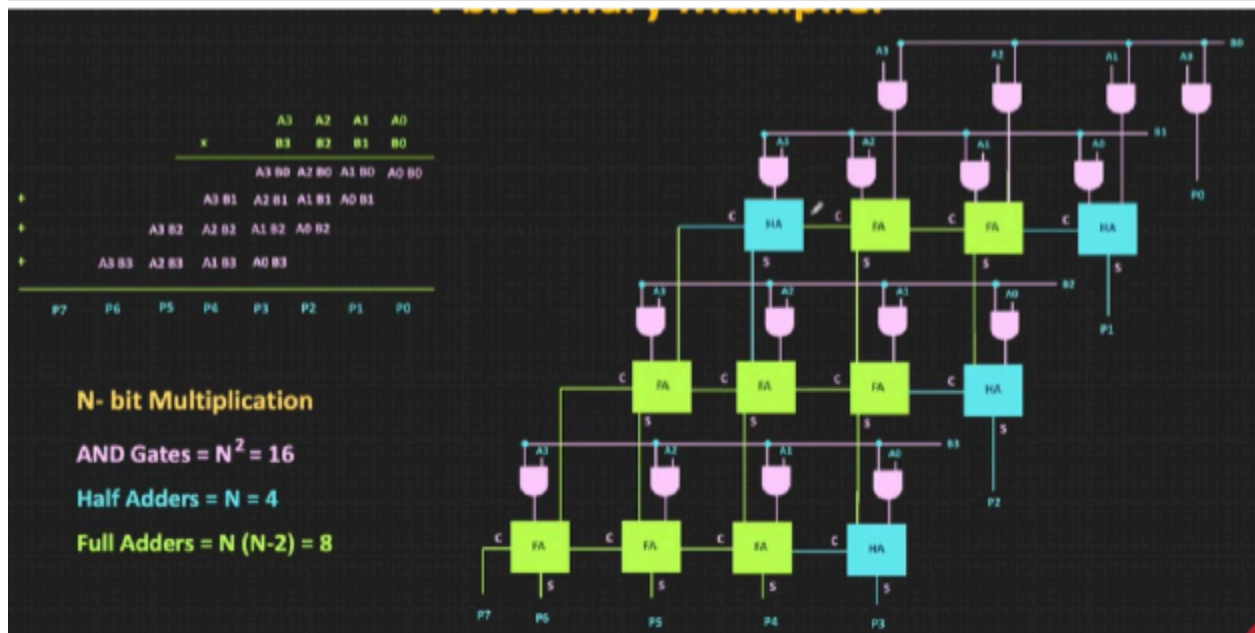
Each bit of multiplicand is multiplied with each bit of multiplier:

This results in **16 partial products** using AND gates.

2- Bit binary multiplier:



4- bit Binary multiplier



Required components:

- AND gates
- Half adders (for first-level addition)
- Full adders (for multi-level addition with carry)
- Switches (inputs)
- LEDs (outputs)
- Wires

Circuit description:

1. Generate partial products using AND gates
2. Shift each partial product according to bit position
3. Add partial products using half and full adders
4. Final output is an 8-bit result

PROCEDURE

1. Open DEEDS simulator
2. Input two 4-bit numbers A and B
3. Use AND gates to generate partial products
4. Use adders to sum the partial products
5. Connect outputs to LEDs
6. Verify output for different inputs

Paste screenshot of circuit diagram implemented on Deeds below:

Truth table

A (4-bit)	B (4-bit)	product (8-bit)
0001	0010	00000010
0011	0010	00000110
0100	0011	00001100
1111	1111	11100001

Q1. What is a partial product?

- A partial product is an intermediate result generated during the multiplication process. In binary multiplication, it is the result of multiplying the entire multiplicand by a single bit of the multiplier. If the multiplier bit is 1, the partial product is the multiplicand itself; if it is 0, the partial product is all zeros.

Q2. Why do we shift in multiplication?

- Shifting aligns the partial products according to the weight (positional value) of the multiplier bit being used. Each successive bit to the left in the multiplier represents a higher power of 2, so the corresponding partial product must be shifted left to ensure that when all partial products are eventually added, they represent their correct place values.

Q3. Difference between serial and parallel multiplier?

- Serial Multiplier: Processes bits sequentially (one at a time). It uses less hardware (typically one adder and shift registers) but is slower because it requires multiple clock cycles to complete one multiplication.
- Parallel Multiplier: Processes all bits simultaneously using a combinational circuit (like an array of adders). It is much faster but requires significantly more logic gates and space on a chip.

Q4. Why are adders needed?

a. Binary multiplication is essentially a process of repeated addition. Once the partial products are generated for each bit of the multiplier, adders are required to sum these intermediate values together to produce the final product.

CONCLUSION (What have you learnt from this experiment?):

Through this experiment, I have learned the fundamental logic behind digital multiplication, which relies on the "and-then-add" principle. I observed how 4-bit inputs generate an 8-bit output to accommodate the maximum possible product ($15 \times 15 = 225$). Furthermore, I understood the critical role of shifting and successive addition in translating simple logic gate operations into complex arithmetic calculations.

Date: 08 May 2026

Lab # 09 TO UNDERSTAND THE WORKING OF MAGNITUDE COMPARATOR CIRCUIT

OBJECTIVES:

- O1: Understand the working of a magnitude comparator
- O2: Design 1-bit and multi-bit comparators using logic gates
- O3: Implement comparator circuits using Boolean expressions
- O4: Verify outputs using truth tables and simulation tools

BACKGROUND THEORY

A magnitude comparator is a combinational circuit used to compare two binary numbers and determine their relative magnitude.

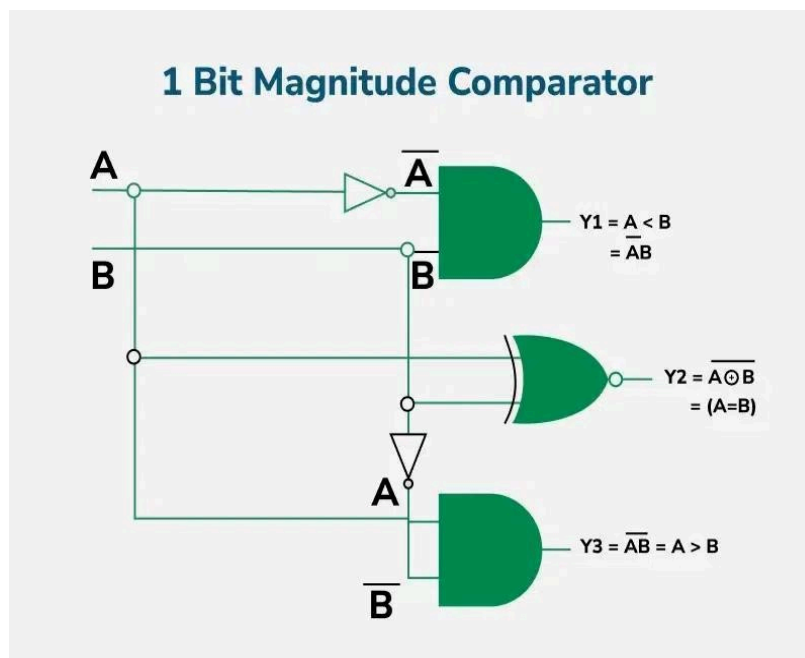
It produces three outputs:

- $A > B$ (A is greater than B)
- $A = B$ (A is equal to B)
- $A < B$ (A is less than B)

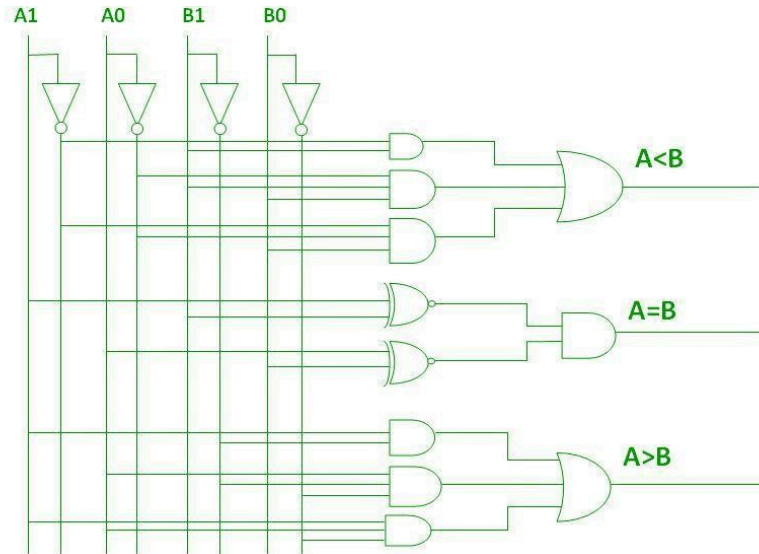
Magnitude comparators are widely used in:

- Digital systems
- Microprocessors
- Decision-making circuits

1- Bit Magnitude Comparator:



2- bit Magnitude Comparator:



2-bit Magnitude Comparator (Concept) For:

- $A = A_1A_0$
- $B = B_1B_0$

Comparison starts from the most significant bit (MSB):

- If $A_1 \neq B_1 \rightarrow$ decision made
- If $A_1 = B_1 \rightarrow$ compare A_0 and B_0

Required components:

- Digital trainer kit / breadboard
- Logic gates ICs (AND, OR, NOT, XNOR)
- Wires and power supply
- Simulation software (e.g., Deeds, Logisim or Proteus)

Procedure:

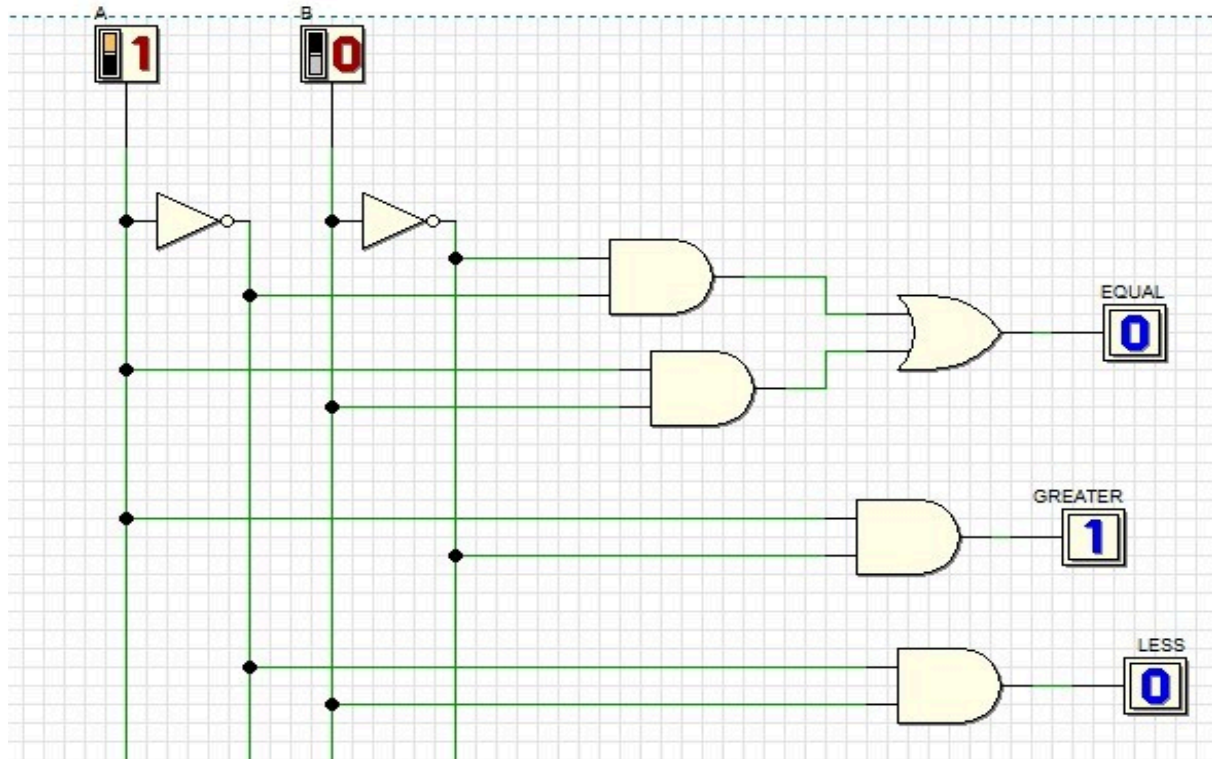
Part A: 1-bit Comparator

1. Implement Boolean expressions using logic gates
2. Connect inputs A and B using switches
3. Observe outputs: $A > B$ $A = B$ $A < B$
4. Verify with truth table

Part B: 2-bit Comparator

1. Design logic using combinations of 1-bit comparators
2. Implement circuit
3. Test all input combinations
4. Record outputs

Paste screenshot of circuit diagram implemented on Deeds below:



TRUTH TABLE: FOR 1-BIT MAGNITUDE COMPARATOR

A	B	A > B	A = B	A < B
0	0	0	1	0
0	1	0	0	1
1	0	1	0	0
1	1	0	1	0

Q1. WHY IS MSB COMPARED FIRST IN MULTI-BIT COMPARATORS?

A. THE MOST SIGNIFICANT BIT (MSB) DETERMINES THE MAGNITUDE. IF $A_3 > B_3$, THEN A IS DEFINITELY GREATER THAN B , REGARDLESS OF THE OTHER BITS. LOWER BITS ARE ONLY CHECKED IF THE HIGHER BITS ARE EQUAL.

Q2. CAN YOU DESIGN A 4-BIT COMPARATOR USING 1-BIT COMPARATORS? EXPLAIN.

A. YES, BY CASCADING. EACH 1-BIT STAGE COMPARES ITS BITS AND PASSES THE RESULT TO THE NEXT. THE "GREATER" OR "LESS" RESULT OF A HIGHER BIT OVERRIDES ALL LOWER BITS; THE "EQUAL" SIGNAL ACTS AS AN ENABLE FOR THE NEXT STAGE.

Q3. WHAT IS THE ROLE OF XNOR GATE IN EQUALITY DETECTION?

A. AN XNOR GATE OUTPUTS 1 ONLY WHEN INPUTS MATCH (0,0 OR 1,1). IN A COMPARATOR, ONE XNOR IS USED PER BIT PAIR. IF EVERY XNOR OUTPUTS 1, THE NUMBERS ARE IDENTICAL.

Q4. HOW ARE COMPARATORS USED IN REAL-LIFE DIGITAL SYSTEMS?

- PROCESSORS: TO EXECUTE "IF/ELSE" LOGIC AND BRANCHING.
- CONTROL SYSTEMS: THERMOSTATS COMPARING CURRENT TEMP TO A SET POINT.
- DIGITAL LOCKS: COMPARING AN INPUT PIN TO A STORED CODE.
- NETWORKING: MATCHING IP ADDRESS SEGMENTS FOR ROUTING.

CONCLUSION (What have you learnt from this experiment?):

THROUGH THIS EXPERIMENT, I SUCCESSFULLY IMPLEMENTED A 1-BIT MAGNITUDE COMPARATOR AND UNDERSTOOD THE LOGIC FOR MULTI-BIT EXPANSION.

KEY LEARNINGS:

- **LOGIC GATES:** I VERIFIED THAT AND/NOT GATES DETERMINE INEQUALITY, WHILE THE XNOR GATE IS THE ESSENTIAL COMPONENT FOR DETECTING EQUALITY.
- **BIT PRIORITY:** I LEARNED THAT COMPARISON MUST START AT THE MSB, AS HIGHER-ORDER BITS DICTATE THE RESULT UNLESS THEY ARE EQUAL.
- **CASCADING:** I UNDERSTOOD THAT MULTI-BIT COMPARATORS ARE BUILT BY CASCADING 1-BIT MODULES, WHERE "EQUAL" SIGNALS FROM HIGHER BITS ENABLE THE COMPARISON OF LOWER BITS.
- **VALIDATION:** THE SIMULATION RESULTS PERFECTLY MATCHED THE TRUTH TABLE, CONFIRMING THE ACCURACY OF THE BOOLEAN EXPRESSIONS IN A PRACTICAL DIGITAL ENVIRONMENT.

Date: 8 MAY 2026

Lab # 10 TO STUDY AND BUILD AN SR LATCH, SR FLIP FLOP AND D-FLIP FLOP

OBJECTIVES:

- O1: To familiarize with the concept of sequential circuits.
- O2: To study the RS Latch, RS Flip-Flop, JK Flip-Flop, and D Flip-Flop
- O3: Understand the concept of memory elements in digital systems
- O4: Differentiate between latches and flip-flops

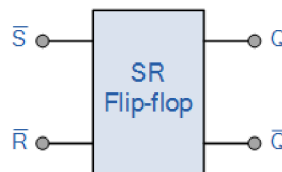
BACKGROUND THEORY

Unlike Combinational Logic circuits that change state depending upon the actual signals being applied to their inputs at that time, Sequential Logic circuits have some form of inherent “Memory” built in to them as they are able to take into account their previous input state as well as those actually present, a sort of “before” and “after” effect is involved with sequential logic circuits. In other words, the output state of a “sequential logic circuit” is a function of the following three states, the “present input”, the “past input” and/or the “past output”. This storage is achieved using **latches and flip-flops**, which are fundamental building blocks of:

- Registers
- Counters
- Memory units
- **Latch** → Level-triggered (depends on input level)
- **Flip-Flop** → Edge-triggered (depends on clock signal)

SR LATCH

The SR (Set-Reset) Latch, can be considered as one of the most basic sequential logic circuit possible. This simple latch is basically a one-bit memory bistable device that has two inputs, one which will “SET” the device (meaning the output = “1”), and is labelled ‘S’ and another which will “RESET” the device (meaning the output = “0”), labelled ‘R’. The reset input resets the flip-flop back to its original state with an output Q that will be either at a logic level “1” or logic “0” depending upon this set/reset condition. The basic circuit symbol of an SR Latch is shown below:



Symbol of an SR Latch

The basic SR Latch can be implemented by using either NAND Gates (NAND Latch) or NOR gates (NOR Latch). Their truth table is given below:

S	R	Q(next)	Description
---	---	---------	-------------

0	0	No Change	Memory
0	1	0	Reset
1	0	1	Set
1	1	Invalid	Forbidden state

RS Flip-Flop (Clocked RS)

Description

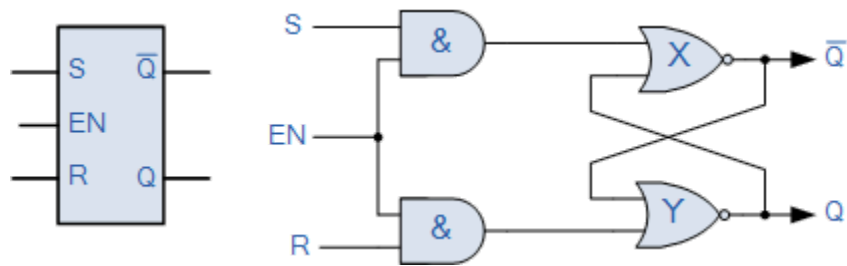
A clocked RS flip-flop improves the RS latch by adding a clock signal (CLK). The output changes only when the clock is active.

Operation

- When CLK = 1 → behaves like RS latch
- When CLK = 0 → output remains unchanged

Limitation

- Still has invalid condition ($S = R = 1$)



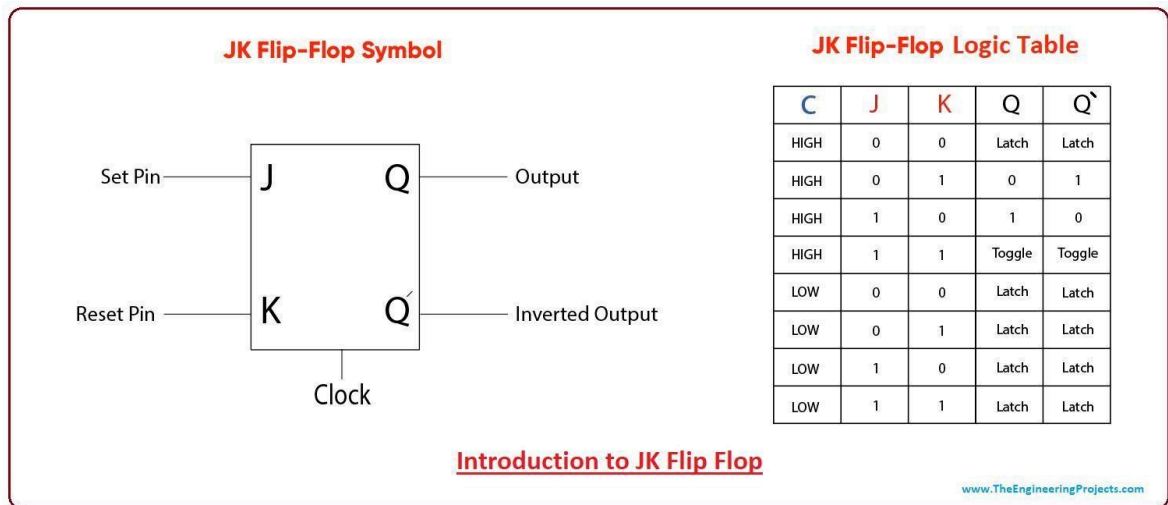
Gated or Clocked SR Latch circuit symbol and gate level Circuit

X	X	0	Qprev	Q'prev <i>Q</i>	
0	0	1	Qprev	Q'prev	
0	1	1	0	1	
1	0	1	1	0	
1	1	1	0	0	Invalid State

Truth Table for Gated SR NOR Flip Flop

JK FLIP FLOP

THE JK FLIP-FLOP REMOVES THE INVALID STATE OF RS FLIP-FLOP.



X	X	0	Qprev	Q'prev
0	0	1	Qprev	Q'prev
0	1	1	0	1
1	0	1	1	0
1	1	1	Q'prev	Q prev
Invalid State				

Truth Table for Gated JK Flip Flop

D-TYPE FLIP FLOP

One way to eliminate the undesirable condition of the indeterminate state in SR Latch is to ensure that the inputs 'S' & 'R' are never equal to '1' at the same time. This is done in the **D-Latch (or D-type Flip Flop)**. The 'D' input goes directly to the 'S' input and its complement is applied to the 'R' input. The resulting Gate level circuit with circuit symbol is shown below:

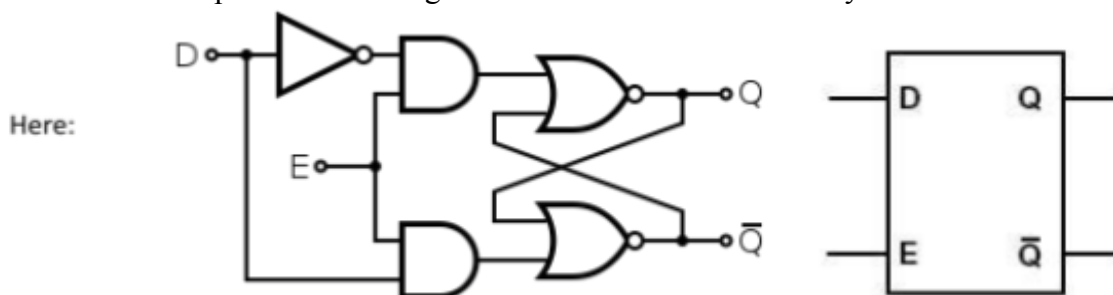


Fig 8.6:

Gated D-Type Flip Flop gate level Circuit and circuit symbol

The Truth Table for Gated SR Flip Flop is as follows:

37

X	0	X	X
1	1		
0	1		

Truth Table for Gated D-Type Flip Flop

PROCEDURE

PART A: RS LATCH

1. OPEN DEEDS
 2. PLACE NOR/NAND GATES
 3. CREATE FEEDBACK CONNECTIONS
 4. APPLY INPUTS S AND R
 5. OBSERVE Q AND $\bar{Q}$
-

PART B: RS FLIP-FLOP

1. ADD CLOCK SIGNAL TO RS LATCH
 2. APPLY DIFFERENT INPUT COMBINATIONS
 3. OBSERVE OUTPUT ONLY DURING CLOCK ACTIVATION
-

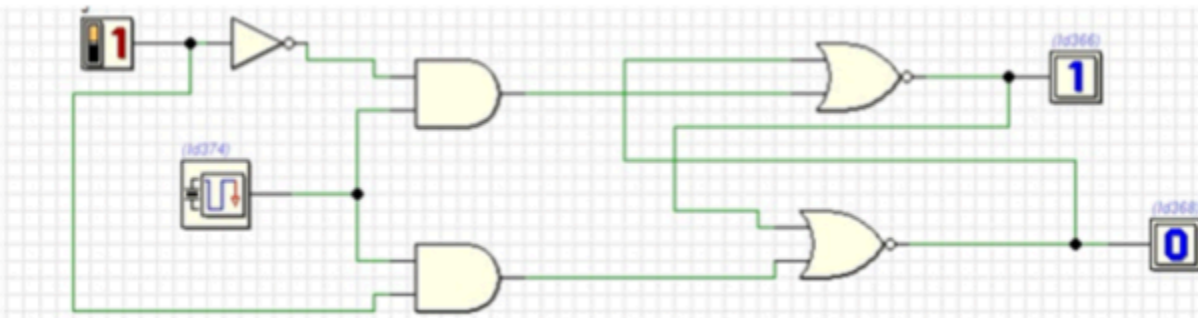
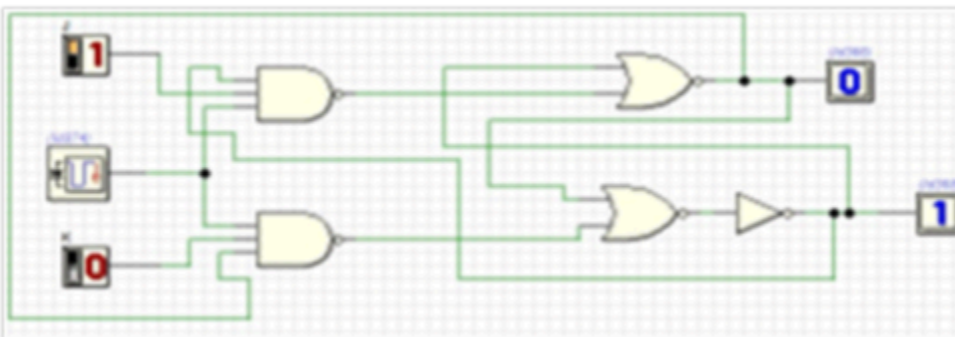
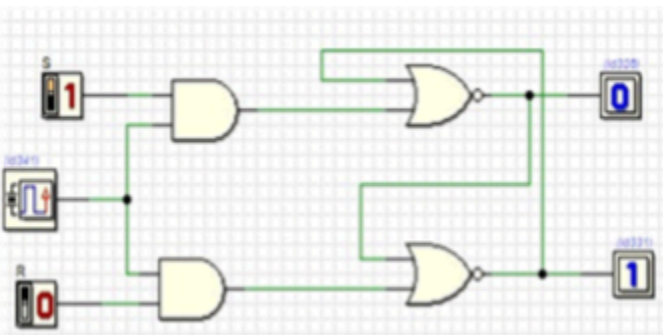
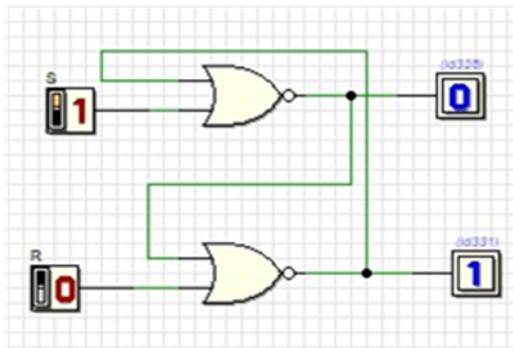
PART C: JK FLIP-FLOP

1. DESIGN JK FLIP-FLOP USING LOGIC GATES
 2. APPLY ALL INPUT COMBINATIONS
 3. VERIFY TOGGLE CONDITION ($J = K = 1$)
-

PART D: D FLIP-FLOP

1. IMPLEMENT D FLIP-FLOP
2. APPLY CLOCK SIGNAL
3. VERIFY OUTPUT FOLLOWS INPUT

PASTE SCREENSHOT OF YOUR IMPLEMENTED CIRCUITS BELOW:



CONCLUSION:

This laboratory experiment successfully demonstrated the design and functional verification of various sequential logic circuits, including the **SR Latch**, **SR Flip-Flop**, **JK Flip-Flop**, and **D**

Flip-Flop. Through implementation in the DEEDS simulator, the following key principles were observed:

- **Objective Accomplished:** The lab successfully demonstrated how sequential circuits use feedback loops to store information as 1-bit memory elements.
- **Triggering Differences:** We identified that **latches** respond to input levels (level-triggered), while **flip-flops** respond to clock signals (edge-triggered).
- **SR Latch & Flip-Flop:** These basic units provide memory but suffer from an "Invalid" state when both inputs are activated simultaneously.
- **JK Flip-Flop:** This design overcomes the SR limitation by allowing the output to **Toggle** when both J and K are high, removing the invalid condition.
- **D Flip-Flop:** By using a single data input and an inverter, this circuit ensures inputs are never identical, effectively eliminating indeterminate states and forcing the output to follow the input.
- **Practical Application:** These components are fundamental building blocks for more complex digital systems, including registers, counters, and memory units.

Name : Syed Sheharyar Tariq

Roll No: 25F-AI-077

Section : A2 / B

Lab # 11 TO STUDY AND REALIZE SHIFT REGISTER.

OBJECTIVES:

O1: To familiarize with the Register concept in circuits.

O2: To study the SISO, PIPO, SIPO, PISO.

BACKGROUND THEORY

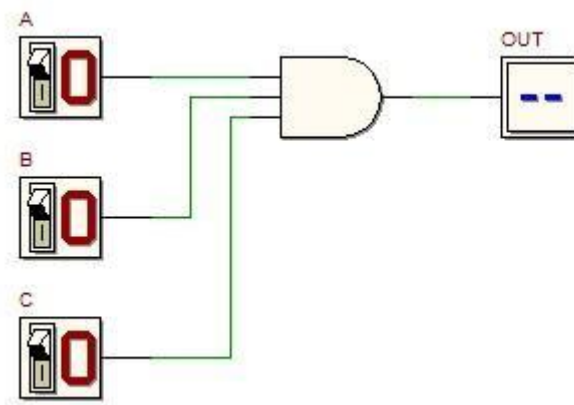
A flip-flop stores 1-bit of digital information. It is also referred to as 1-bit register. An array of flip-flops is required to store the no. of bits. This is called register. The data can be entered into or retrieved from the register. So depending on the way how the data can be entered or retrieved there are four possible modes of operation.

- 1)Serial in serial out (SISO)
- 2)Serial in parallel out (SIPO)
- 3)Parallel in serial out (PISO)
- 4)Parallel in parallel out (PIPO)

Registers can be designed using J-K or S-R as D-type flip-flops and are also available as MSI devices. The register which can be operated in all possible modes & also the shifting of data is possible in both the directions is called Shift Register.

Digital Circuit Simulator:

Deeds software is also used for this lab. In this introductory exercise you will test the simple logic network represented in the figure below and, at the same time, you will gain confidence with the **Digital Circuit Simulator (d-DcS)** of the *Deeds*. To open the file in the **d-DcS**, just click on the figure:



Next step is to check the behavior of the network. You will start the **functional simulation**

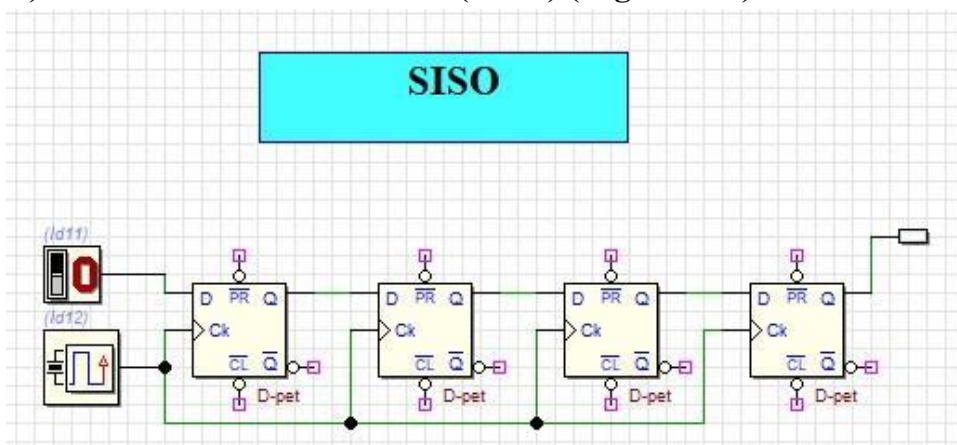
(Interactive Animation) of the network by clicking, on the **d-DcS toolbar**, the command . Now the three input switches **A**, **B** and **C** can be toggled and the gate's output **OUT** will change accordingly. We suggest that you draw the truth table for a three variable boolean function and then fill the output column with the data resulting from the simulation.

Last task requested is the **timing simulation** of the same network. You start the timing simulation of the network by clicking, on the **d-DcS toolbar**, the command . The input values must be drawn directly on the **timing diagram window**. You should define the values versus time of the three inputs, such as all the possible combinations of **A**, **B** and **C** are tested.

CIRCUIT SIMULATION DIAGRAM:

TRUTH TABLE:

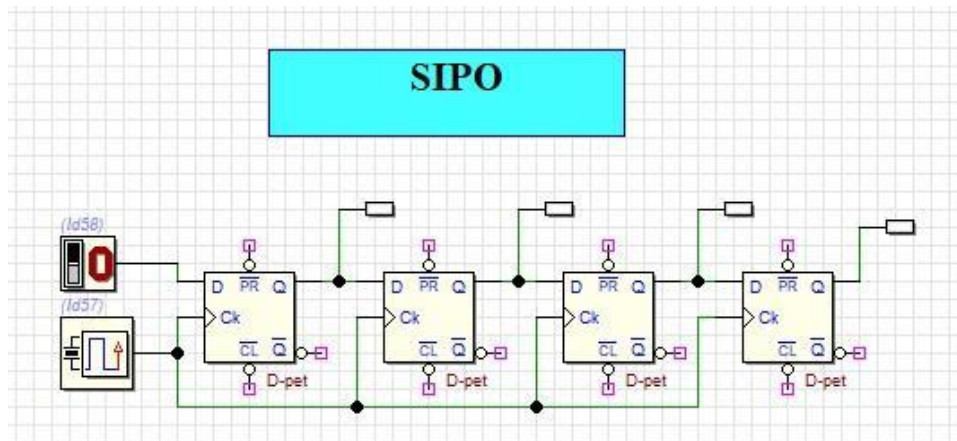
1) SERIAL IN SERIAL OUT (SISO) (Right Shift)



Serial i/p data	Shift Pulses	Q _A	Q _B	Q _C	Q _D
Initial	0	0	0	0	0
1	1	1	0	0	0
1	2	1	1	0	0
0	3	0	1	1	0
1	4	1	0	1	1
0	5	0	1	0	1
0	6	0	0	1	0
0	7	0	0	0	1
0	8	0	0	0	0

Table 9.1

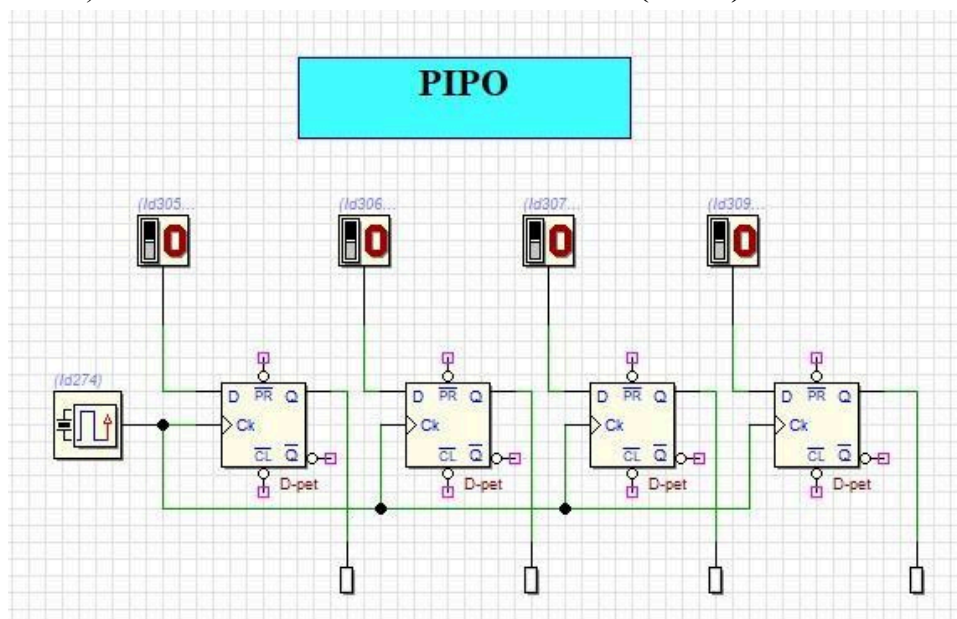
2): SERIAL IN PARALLEL OUT (SIPO)



Serial i/p data	Shift Pulses	Q_A	Q_B	Q_C	Q_D
Initial	0	0	0	0	0
1	1	1	0	0	0
0	2	0	1	0	0
1	3	1	0	1	0
1	4	1	1	0	1

Table 9.2

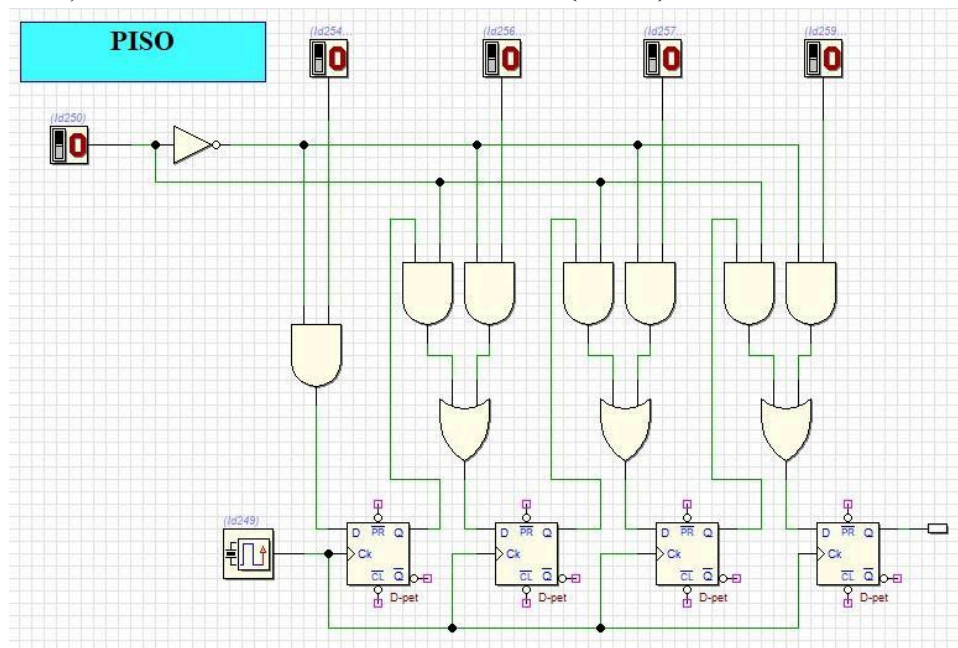
3): PARALLEL IN PARALLEL OUT (PIPO)



Clock Input Terminal	Shift Pulses	Q _A	Q _B	Q _C	Q _D
↓	0	0	0	0	0
↓	1	1	0	1	1

Table 9.3

4): PARALLEL IN SERIAL OUT (PISO)



Clock Input Terminal	Shift Pulses	Q _A	Q _B	Q _C	Q _D
↓ (Load=1)	0	1	0	1	1
↓ (Load=0)	1	0	1	0	1
↓ (Load=0)	2	0	0	1	0
↓ (Load=0)	3	0	0	0	1
↓ (Load=0)	4	0	0	0	0
↓ (Load=0)	5	0	0	0	0

Table 9.4

PROCEDURE:

- Check all the components for their working.
- Insert the appropriate IC into the IC base.
- Make connections as shown in the circuit diagram.
- Verify the Truth Table and observe the outputs.

CONCLUSION (What have you learnt from this experiment?):

From this experiment, we have learnt that:

1. A Shift Register is a digital circuit that can store and shift data. It consists of a cascade of flip-flops (typically D-type) connected in series, where the output of one flip-flop is connected to the input of the next.
2. There are four types of shift registers based on data input and output methods:
 - SISO (Serial In Serial Out): Data is entered one bit at a time and read out one bit at a time. It acts as a temporary storage device and introduces delay.
 - SIPO (Serial In Parallel Out): Data is entered serially but can be read all at once in parallel form. It converts serial data to parallel data.
 - PISO (Parallel In Serial Out): Data is loaded in parallel but read out serially. It converts parallel data to serial data.
 - PIPO (Parallel In Parallel Out): Data is loaded and read simultaneously in parallel form. It acts as a simple data storage register without shifting.
3. Shift registers are fundamental building blocks in digital systems and are used in applications such as data storage, data transfer, serial-to-parallel conversion, parallel-to-serial conversion, and delay elements.
4. The Deeds software provides an effective platform for simulating and verifying the behavior of shift registers before actual hardware implementation.

Name: Syed Sheharyar Tariq

Roll No: 25F-AI-077

Section: A2/B

Lab # 12 OPEN ENDED LAB II – SEQUENTIAL CIRCUIT

OBJECTIVES:

O1: To apply the concepts of sequential logic circuits in solving real-world problems.

O2: To design basic sequential systems using flip-flops, counters, and registers.

O3: To implement timing-based and step-based digital systems using hardware simulation tools.

BACKGROUND THEORY:

Sequential circuits are digital systems whose outputs depend on both **current inputs and previous states**. Unlike combinational circuits, sequential circuits have memory elements that store past information.

The main building blocks of sequential circuits include:

- **Flip-Flops (SR, JK, D, T)** → basic memory elements
- **Counters** → used to count events or steps
- **Registers** → used to store and shift data
- **Clock signals** → control timing and sequence of operations

Sequential circuits are widely used in real-life applications such as:

- Traffic light systems
- Vending machines
- Digital clocks
- Simple control systems
- Security lock systems

LAB TASKS:

The main goal of this lab is to design sequential circuits for real-world scenarios using basic sequential components. **You can select any 1 scenario from following.**

For every scenario:

- Understand the problem
- Identify required steps or stages
- Use **flip-flops and counters** to represent sequence
- Design logic circuits using simulation software
- Verify system behavior using clock cycles

SCENARIO 1: TRAFFIC LIGHT CONTROL SYSTEM

Design a sequential circuit for a simple traffic light system at a road crossing.

System Requirements:

- Three lights: Red, Yellow, Green

- Lights change in a fixed sequence:
 - Green → Yellow → Red → Green
- Each light remains ON for a fixed number of clock pulses
- Use counters to control timing of each state **Tasks:**
 1. Assign binary representation to each light condition
 2. Use a counter to generate timing delays
 3. Design logic circuit using flip-flops and gates
 4. Implement and simulate the circuit
 5. Observe correct light sequencing with clock pulses

SCENARIO 2: SIMPLE VENDING MACHINE (COIN BASED SYSTEM)

Design a sequential circuit for a vending machine that dispenses a product worth Rs. 20.

System Requirements:

- Accept Rs. 5 and Rs. 10 coins
- Keep track of total inserted amount using flip-flops/registers
- When total reaches Rs. 20 → product is dispensed
- Reset system after dispensing **Tasks:**
 1. Represent amounts using binary states
 2. Use flip-flops to store accumulated value
 3. Design addition logic for coin inputs
 4. Generate output signal when total = 20
 5. Simulate system behavior

SCENARIO 3: PASSWORD-BASED DIGITAL LOCK SYSTEM

Design a sequential circuit for a 3-bit password lock system.

System Requirements:

- A fixed password sequence (example: 1-0-1)
- Input is given bit by bit using a switch
- System checks sequence step by step
- If correct sequence is entered → unlock signal ON
- If wrong input occurs → system resets

Tasks:

1. Store input bits using flip-flops
2. Use clock pulses for step-by-step input checking
3. Design logic to compare input sequence
4. Generate unlock output
5. Simulate correct and incorrect inputs

SCENARIO 4: DIGITAL COUNTER-BASED DISPLAY SYSTEM

Design a sequential circuit that displays numbers from 0 to 9 repeatedly.

System Requirements:

- Use a counter to generate sequence 0–9
- Reset counter after reaching 9
- Each count should appear after a clock pulse
- Output can be shown using LEDs or 7-segment display **Tasks:**

1. Design a 4-bit counter using flip-flops
2. Define reset condition after 9
3. Connect output to display system
4. Simulate counting sequence
5. Observe continuous cycle operation

EXPECTED OUTCOMES:

After completing this lab, students will be able to:

- Design sequential circuits using flip-flops and counters
- Implement timing-based digital systems
- Understand practical applications of sequential logic
- Simulate real-world digital control systems
- Develop problem-solving skills in digital design

TOOLS REQUIRED:

- Logisim / Logisim Evolution
- Proteus Design Suite / Multisim
- Basic digital logic simulation software

HANDOUT REQUIREMENTS:

- Theory and circuit diagram of the Logic
- Implementation on the Simulator
- Screenshot and Results
- Conclusion

SCENARIO 4: DIGITAL COUNTER-BASED DISPLAY SYSTEM

PROBLEM STATEMENT

Design a sequential circuit that displays numbers from 0 to 9 repeatedly.

SYSTEM REQUIREMENTS

- Use a counter to generate sequence 0–9
- Reset counter after reaching 9
- Each count should appear after a clock pulse
- Output can be displayed using LEDs or 7-segment display

TASKS

1. Design a 4-bit counter using flip-flops
2. Define reset condition after 9
3. Connect output to display system
4. Simulate counting sequence
5. Observe continuous cycle operation

THEORY OF DIGITAL COUNTER

A digital counter is a sequential circuit that counts clock pulses. In this experiment, a 4-bit counter is designed using flip-flops to generate binary numbers from 0000 to 1001.

Each clock pulse changes the state of the counter. After reaching decimal 9 (1001), the counter resets back to 0000 and repeats the sequence continuously.

COMPONENTS REQUIRED

- Toggle Switch
- Clock Pulse
- T Flip-Flops / JK Flip-Flops
- AND Gates
- NOT Gates
- LEDs
- Reset Logic
- Connecting Wires

BINARY COUNTING SEQUENCE

Decimal Binary

0	0000
1	0001
2	0010
3	0011
4	0100
5	0101
6	0110
7	0111
8	1000
9	1001

RESET CONDITION

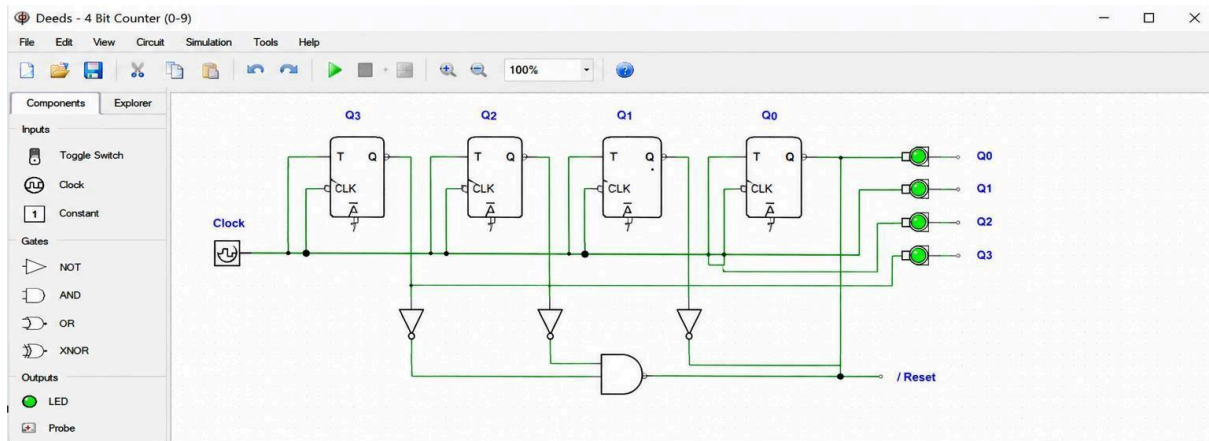
The counter resets after reaching:

[1001 = 9]

After this state, the output becomes:

[0000]

CIRCUIT DIAGRAM



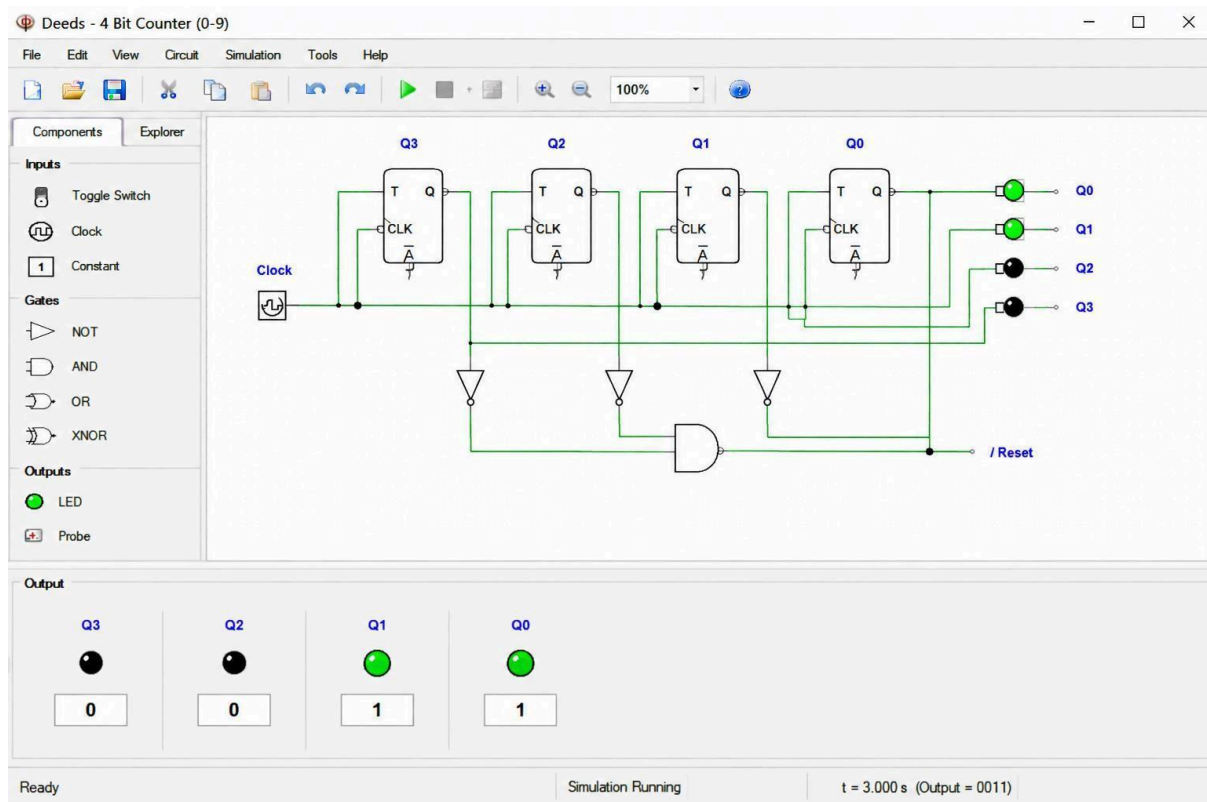
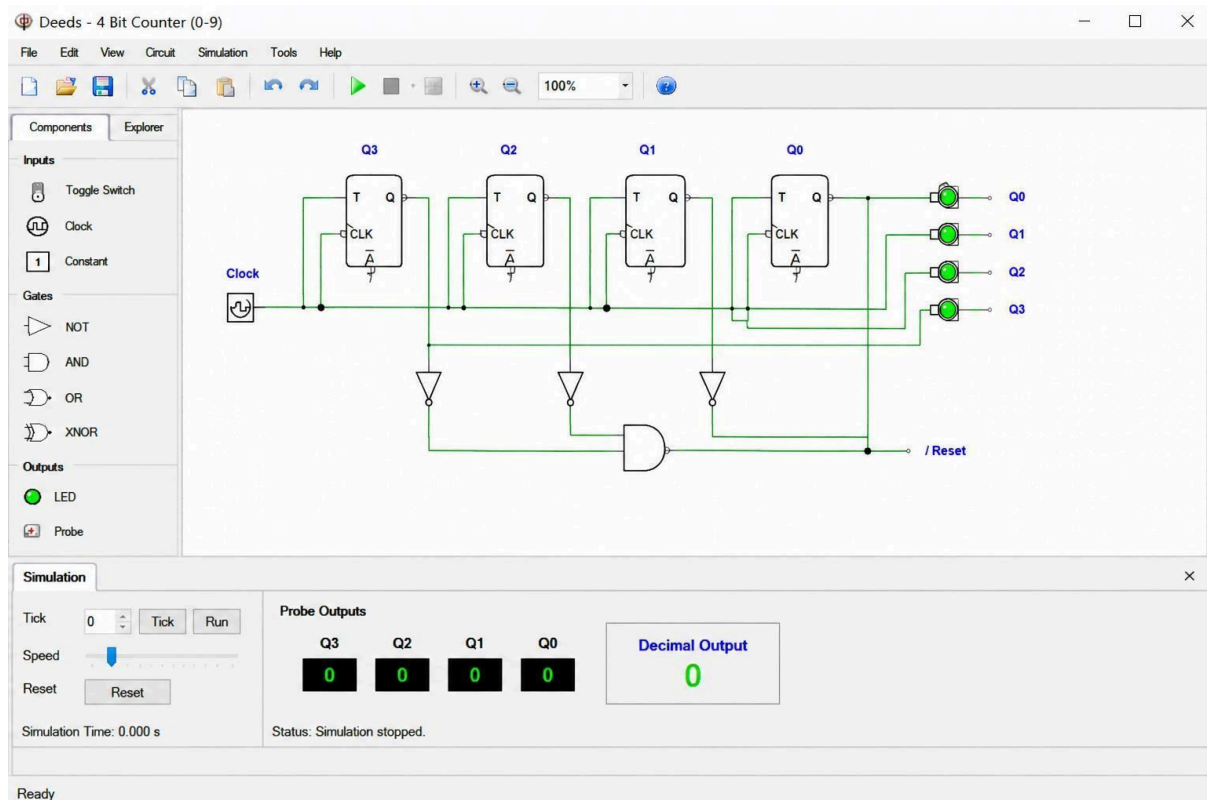
IMPLEMENTATION ON SIMULATOR

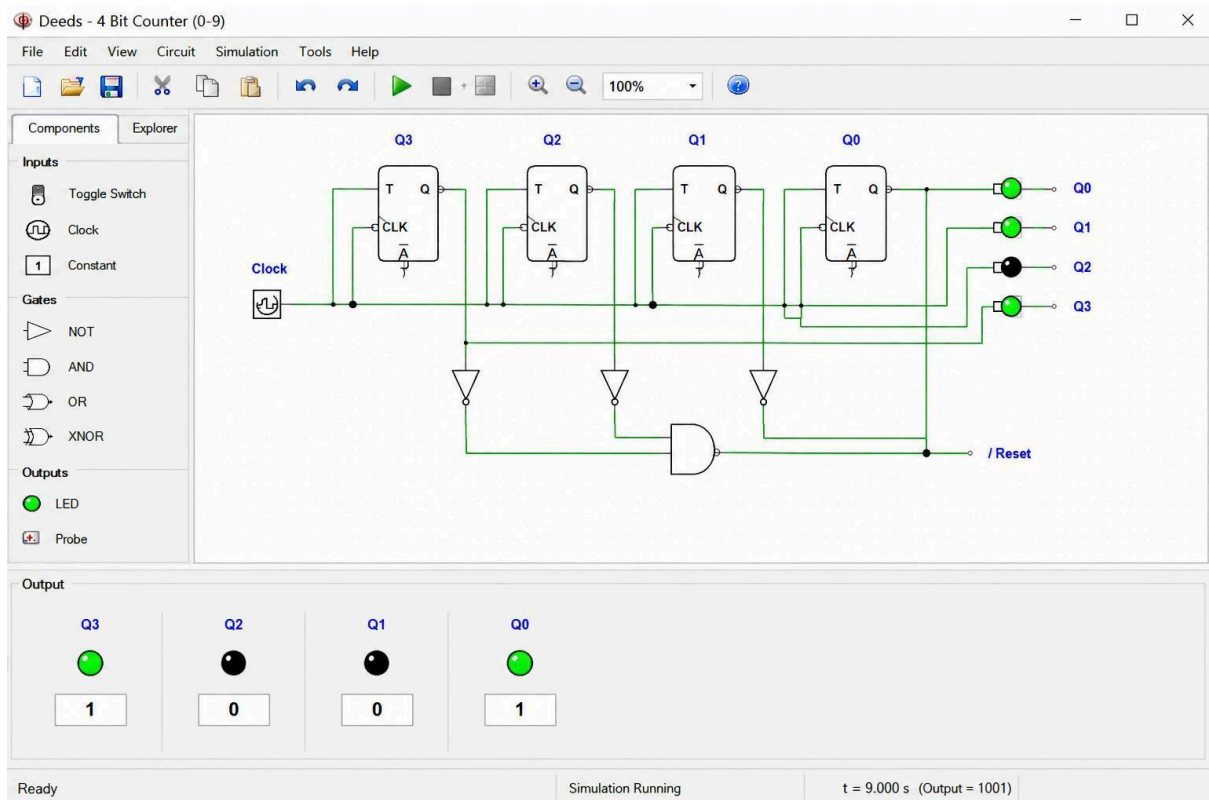
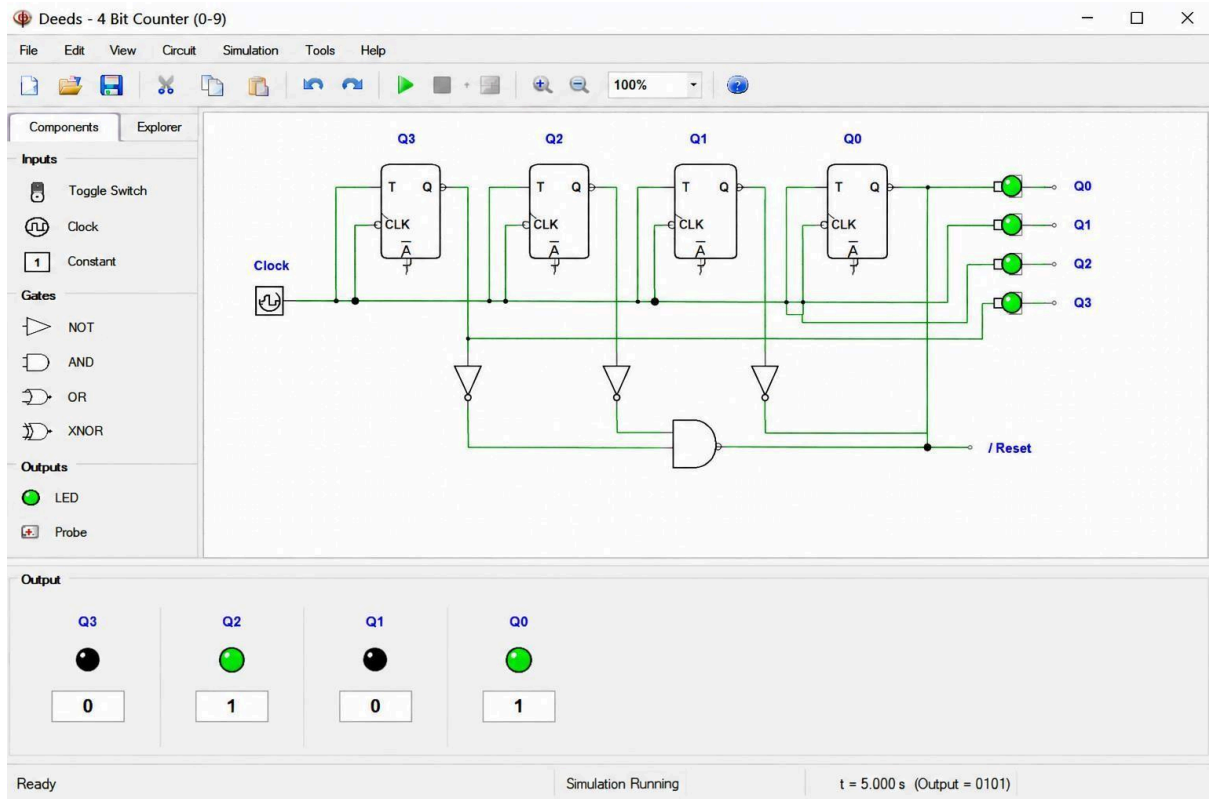
The circuit was implemented using Deeds digital logic simulator.

A 4-bit sequential counter was designed using flip-flops and clock pulses. LEDs were connected to outputs to observe binary counting states.

The counter generated outputs from 0000 to 1001 successfully.

SIMULATION RESULTS





OBSERVATION TABLE

Clock Pulse Output

1	0000
2	0001
3	0010
4	0011
5	0100
6	0101
7	0110
8	0111
9	1000
10	1001
Next	0000

RESULT

The 4-bit sequential counter successfully counted numbers from 0 to 9 using clock pulses. After reaching decimal 9, the counter automatically reset to 0 and repeated the counting sequence.

CONCLUSION

In this experiment, a sequential counter circuit was designed and implemented using flip-flops. The system successfully generated counting states from 0 to 9 and demonstrated the practical working of sequential logic circuits. The experiment also helped in understanding clock pulses, memory elements, and counter-based digital systems.

